



**POLITECNICO**  
**MILANO 1863**

SCUOLA DI INGEGNERIA INDUSTRIALE  
E DELL'INFORMAZIONE

A PROJECT FOR DESIGN AND IMPLEMENTATION OF MOBILE  
APPLICATIONS COURSE

# DimApp

Author:

**Pietro Ammaturo - Andrès Bonilla**

Teacher:

**Prof. Luciano Baresi**

Academic Year:

**2024-25**

# Contents

|                                                      |          |
|------------------------------------------------------|----------|
| <b>Contents</b>                                      | <b>1</b> |
| <b>1 Introduction</b>                                | <b>3</b> |
| 1.1 Purpose . . . . .                                | 3        |
| 1.2 Requirements . . . . .                           | 3        |
| 1.3 Features Implemented . . . . .                   | 4        |
| 1.3.1 Login and Registration . . . . .               | 4        |
| 1.3.2 Deposit and Withdrawal . . . . .               | 4        |
| 1.3.3 Add contact . . . . .                          | 4        |
| 1.3.4 Transfers to contacts . . . . .                | 4        |
| 1.3.5 Upgrade account to professional . . . . .      | 4        |
| 1.3.6 Shop: create payment request . . . . .         | 5        |
| 1.3.7 Shop: move funds to personal account . . . . . | 5        |
| 1.3.8 QR code scanner . . . . .                      | 5        |
| 1.3.9 Map with shops position . . . . .              | 5        |
| 1.3.10 Notifications . . . . .                       | 5        |
| <b>2 Application Architecture</b>                    | <b>6</b> |
| 2.1 Data Management . . . . .                        | 6        |
| 2.1.1 Data Model Design . . . . .                    | 6        |
| 2.1.2 User Collection . . . . .                      | 6        |
| 2.1.3 Shop collection . . . . .                      | 7        |
| 2.2 Database Implementation . . . . .                | 7        |
| 2.2.1 Cloud Firestore Schema . . . . .               | 7        |
| 2.3 Device Access . . . . .                          | 10       |
| 2.4 External Services . . . . .                      | 10       |
| 2.4.1 Firebase Authentication . . . . .              | 10       |
| 2.4.2 Google Sign-In . . . . .                       | 11       |
| 2.4.3 Facebook Sign-In . . . . .                     | 11       |
| 2.4.4 Maps . . . . .                                 | 12       |
| 2.4.5 Google Cloud Functions . . . . .               | 12       |
| 2.4.6 Expo Push Notifications . . . . .              | 12       |
| 2.5 Dependencies . . . . .                           | 13       |
| 2.6 Component Architecture . . . . .                 | 14       |
| 2.6.1 Layout Components . . . . .                    | 15       |
| 2.6.2 Section Components . . . . .                   | 15       |
| 2.6.3 Listable Components . . . . .                  | 15       |

|          |                                  |           |
|----------|----------------------------------|-----------|
| 2.6.4    | Modal Components . . . . .       | 15        |
| 2.6.5    | Input Components . . . . .       | 16        |
| 2.6.6    | Base Components . . . . .        | 16        |
| 2.7      | Sequence Diagrams . . . . .      | 16        |
| 2.7.1    | Sign Up with Google . . . . .    | 17        |
| 2.7.2    | Add Contact . . . . .            | 18        |
| 2.7.3    | Pay using QR . . . . .           | 19        |
| 2.7.4    | Add Shop Flow . . . . .          | 20        |
| <b>3</b> | <b>User Interface (UI)</b>       | <b>22</b> |
| 3.1      | Navigation . . . . .             | 22        |
| 3.2      | UI Design Choices . . . . .      | 22        |
| 3.3      | Smartphone UI . . . . .          | 23        |
| 3.3.1    | Login and Registration . . . . . | 23        |
| 3.3.2    | Dashboard . . . . .              | 24        |
| 3.3.3    | Me . . . . .                     | 25        |
| 3.3.4    | Add Shop . . . . .               | 26        |
| 3.3.5    | Contacts . . . . .               | 27        |
| 3.3.6    | Chat . . . . .                   | 28        |
| 3.3.7    | Scanner . . . . .                | 30        |
| 3.3.8    | Maps . . . . .                   | 31        |
| 3.4      | Tablet UI . . . . .              | 34        |
| 3.4.1    | Dashboard . . . . .              | 35        |
| 3.4.2    | Shop . . . . .                   | 35        |
| 3.4.3    | Contacts & Chat . . . . .        | 36        |
| 3.4.4    | My Shop . . . . .                | 37        |
| 3.4.5    | Me . . . . .                     | 38        |
| 3.4.6    | Other pages . . . . .            | 38        |
| 3.5      | Dark mode . . . . .              | 39        |
| <b>4</b> | <b>Testing</b>                   | <b>42</b> |
| 4.1      | Testing Campaign . . . . .       | 42        |
| 4.2      | Unit Test . . . . .              | 42        |
| 4.3      | Component Test . . . . .         | 43        |
| 4.4      | Integration Test . . . . .       | 45        |
| 4.5      | User Test . . . . .              | 48        |

# 1 | Introduction

## 1.1. Purpose

DimApp is an innovative mobile application designed to simplify money transfers between individuals, while also offering an efficient payment solution for small and medium-sized businesses. Within the app, users can send and receive money quickly and securely, making financial transactions smoother and more immediate.

The user interface is intuitive and user-friendly, inspired by a chat-style layout that displays transactions as conversations—making payment interactions feel natural and easy to follow. A built-in QR code feature allows users to make payments or request money with a simple scan, removing the need for manual data entry.

With DimApp, exchanging money becomes a fast, seamless, and social experience—right inside the app.

## 1.2. Requirements

The following table contains all the requirements that the application must satisfy.

| Code | Description                                                                                                               |
|------|---------------------------------------------------------------------------------------------------------------------------|
| R1   | The user must be able to register in the application                                                                      |
| R2   | The user must be able to log in to the application.                                                                       |
| R3   | The user must be able to view their current balance.                                                                      |
| R4   | The user must be able to deposit funds into their account.                                                                |
| R5   | The user must be able to withdraw funds from their account.                                                               |
| R6   | The user must be able to view transactions ordered from newest to oldest.                                                 |
| R7   | The user must be able to generate a QR code containing their user information.                                            |
| R8   | The user must be able to add a new contact using their email address.                                                     |
| R9   | The user must be able to add a new contact scanning the personal QR code generated by other.                              |
| R10  | The user must be able to view the list of added contacts.                                                                 |
| R11  | The user must be able to transfer funds to a contact.                                                                     |
| R12  | The user must be able to view the full transaction history with a contact.                                                |
| R13  | The user must be able to upgrade their account by associating a shop with it.                                             |
| R14  | The user must be able to see all registered shops on a map within the app.                                                |
| R15  | The user with a shop associated with the account must be able to view all transactions of the shop from newest to oldest. |

| Code | Description                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------------------|
| R16  | The user with a shop associated with the account must be able to generate a QR code for a payment request.             |
| R17  | The user must be able to confirm a payment request by scanning the QR code generated by the shop.                      |
| R18  | The user with a shop associated with the account must be able to check the status of the payment request in real time. |
| R19  | The user should be able to use the application from both smartphones and tablets.                                      |
| R20  | The user should receive notifications about payments and contact addition.                                             |

**Table 1.1: Requirements table**

## **1.3. Features Implemented**

### **1.3.1. Login and Registration**

Users can access the app exclusively through Single Sign-On (SSO), using their existing Google or Facebook accounts as identity providers. This streamlined login process eliminates the need to create and remember new credentials, ensuring a faster, more secure, and user-friendly authentication experience.

### **1.3.2. Deposit and Withdrawal**

This functionality has been simulated, meaning the in-app currency is not linked to real-world payment systems. However, users can deposit and withdraw virtual funds, which has been essential for testing and validating the application's various features.

### **1.3.3. Add contact**

Users can add contacts, similar to how a phone address book works. There are two convenient methods to add a contact: the first method allows them to manually search for a user by entering their email address, while the second method involves scanning a QR code that contains the user's details.

### **1.3.4. Transfers to contacts**

Users can transfer funds exclusively to their added contacts through a chat-like experience, where transactions appear as if they were messages in a conversation, similar to common messaging apps. This approach makes managing connections and tracking transaction history both simple and intuitive.

### **1.3.5. Upgrade account to professional**

Users can upgrade their account to a professional one, enabling them to associate a shop, either physical or digital. The shop functions as an independent entity, maintaining

its own balance and transaction completely separate from the user's personal account. To create a shop, users are required to provide the location and their most recent utility bill.

#### **1.3.6. Shop: create payment request**

To receive a payment, shop owners must create a payment request by generating a QR code that contains all the necessary transaction details. This QR code must then be scanned by the customer using the in-app QR scanner in order to complete the payment. This is the only way for a shop to receive payments within the app, ensuring a fast and streamlined transaction process.

#### **1.3.7. Shop: move funds to personal account**

Shop owners can move funds from the Shop balance to the User balance. In this way, the Shop balance is set to 0 and all the funds are moved to the personal account. This will be considered as transactions on both sides.

#### **1.3.8. QR code scanner**

The application includes a built-in QR code scanner, which plays a key role in its functionality. It enables users to easily add new contacts and approve payment requests from shops, streamlining the overall experience.

#### **1.3.9. Map with shops position**

Users can view all available shops displayed on a map relative to their current location. This feature enhances the user experience by making it easy to discover nearby shops where the app can be used for payments. Clicking on a shop will open that shop's location on google maps.

#### **1.3.10. Notifications**

Users can receive notifications about contact addition and transactions. Such notifications will open the app if clicked.

## 2 | Application Architecture

The application has been designed as a Thick Client App with a Service-Oriented Backend architecture. The client-side is a mobile application developed using React Native, responsible for handling the majority of the presentation and business logic to deliver a seamless and consistent user experience across both iOS and Android platforms from a single codebase. Instead of relying on a single monolithic backend, the application integrates with a range of external services—such as authentication providers, cloud storage, and a NoSQL database—each chosen for their flexibility, scalability, and performance. This distributed approach enables the system to remain modular and maintainable while leveraging best-of-breed solutions for individual backend functions.

### 2.1. Data Management

This section describes how data is structured and managed within the application.

#### 2.1.1. Data Model Design

The adopted solution is a NoSQL database, chosen to enhance performance, a crucial factor in mobile applications. The data model follows a usage-driven approach, structuring the data based on access patterns to maximize efficiency and is stored in two collections: **User** and **Shop**, each containing relevant sub-collections.

Following the NoSQL paradigm, certain data has been deliberately duplicated across documents to optimize query performance. While this introduces redundancy, the trade-off is justified by the significant gains in efficiency for our specific use case. However, managing updates to replicated data, ensuring consistency across all copies, falls outside the scope of this implementation.

#### 2.1.2. User Collection

The **User** collection contains all relevant information such as personal details (e.g., name and surname), account balance, added contacts, and executed transactions. To optimize performance, contacts and transactions are structured as sub-collections, enabling more efficient querying.

### 2.1.3. Shop collection

The **Shop** collection stores all relevant information about each shop, including the owner's email, location, current balance, and executed transactions. To enhance performance, transactions are organized as sub-collections, allowing for more efficient queries.

The decision to create a separate **Shop** collection, rather than nesting it within the **User** collection, was driven by the application's usage requirements. This approach enables querying all shops by location range, which is a key requirement (R14).

## 2.2. Database Implementation

To implement the NoSQL database, the chosen solution is Cloud Firestore, a flexible and scalable platform built on Google Cloud infrastructure.

### 2.2.1. Cloud Firestore Schema

User collection schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "title": "User",
  "properties": {
    "name": { "type": "string" },
    "surname": { "type": "string" },
    "email": { "type": "string" },
    "amount": { "type": "number" },
    "tms_creation": { "type": "string", "format": "date-time" },
    "contacts_ref": {
      "type": "string",
      "description": "Reference to the 'Contacts' subcollection"
    },
    "transactions_ref": {
      "type": "string",
      "description": "Reference to the 'Transactions' subcollection"
    }
  },
  "additionalProperties": false
}
```

The User collection stores all documents containing all relevant information about users, including personal details such as name, surname, and email address, as well as other data like account balance and references to contacts and transactions. Each document in the User collection assumes that the 'email' field is unique.

## User/Contacts sub-collection schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "title": "Contacts",
  "properties": {
    "name": { "type": "string" },
    "surname": { "type": "string" },
    "email": { "type": "string" }
  },
  "additionalProperties": false
}
```

The Contacts sub-collection within the User collection contains all the contacts added by an individual user. Each contact includes information such as name, surname, and email address. As with the User collection, each document in the Contacts sub-collection assumes the 'email' field is unique.

## User/Transactions sub-collection schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "title": "Transactions",
  "properties": {
    "amount": { "type": "number" },
    "description": { "type": "string" },
    "tms": { "type": "string", "format": "date-time" },
    "transactor": {
      "type": "object",
      "properties": {
        "displayName": { "type": "string" },
        "email": { "type": "string" },
        "is_shop": { "type": "boolean" }
      },
      "additionalProperties": false
    },
    "additionalProperties": false
  },
  "additionalProperties": false
}
```

The Transactions sub-collection within the User collection contains all transactions involving the individual user, including both incoming and outgoing payments. Each document stores details such as the amount, description, execution timestamp, and personal information about the counterparty. Since a user's shop shares the same email address, a Boolean field 'is\_shop' is used to distinguish whether the transaction involves the user personally or their shop.

## Shop collection schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "title": "Shop",
  "properties": {
    "shopName": { "type": "string" },
    "email": { "type": "string" },
    "description": { "type": "string" },
    "position": { "type": "object", "additionalProperties": false },
    "balance": { "type": "number" },
    "tms_creation": { "type": "string", "format": "date-time" },
    "transactions_ref": {
      "type": "string",
      "description": "Reference to the 'Transactions' subcollection"
    }
  },
  "additionalProperties": false
}
```

The Shop collection stores all documents containing information about individual shops, including details such as name, description, and location, along with references to related transactions. Each document assumes that the 'email' field is unique, meaning a user can own at most one shop.

## Shop/Transactions sub-collection schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "title": "Transactions",
  "properties": {
    "amount": { "type": "number" },
    "description": { "type": "string" },
    "tms": { "type": "string", "format": "date-time" },
    "transactor": {
      "type": "object",
      "properties": {
        "displayName": { "type": "string" },
        "email": { "type": "string" },
        "is_shop": { "type": "boolean" }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

The Transactions sub-collection within the Shop collection shares the same structure as the User/Transactions sub-collection. It is used to store all information related to transactions involving the shop.

## 2.3. Device Access

Here we list all the device-specific functionalities used in the application.

### Camera

The camera is accessed to scan QR codes in two primary scenarios: adding contacts and making payments at shops. By integrating QR code scanning directly into the app, the user experience becomes faster, safer and more convenient, eliminating the need for manual input or third-party QR scanners.

### Location

Location services are used to determine the device's geographic position. This enables the app to offer location-based features such as displaying nearby shops and allowing users to set their shop's location using their current position.

### Gyroscope

The gyroscope is used to detect the orientation of the device. This information is crucial for adjusting the user interface between portrait and landscape modes, and for accurately rotating the navigation arrow on the map screen. By responding to device orientation changes, the application ensures a more intuitive and visually consistent navigation experience.

### Shared Preferences

Shared Preferences are used to retrieve the application's theme setting based on the user's dark mode preference.

## 2.4. External Services

Our application makes use of several external services in addition to the firebase storage previously shown. These services enable some of the core features while keeping the main code base light and maintainable.

### 2.4.1. Firebase Authentication

Firebase Authentication provides backend services, easy-to-use SDKs, and ready-made UI libraries to authenticate users. It supports authentication using passwords, phone numbers, popular federated identity providers like Google, Facebook and Twitter, and more.

Firebase Authentication integrates tightly with other Firebase services, given that our data management relies on cloud fire-store the choice of Firebase's integrated authentication mechanism is obvious.

### 2.4.2. Google Sign-In

Google Sign-In manages the OAuth 2.0 flow and token lifecycle, simplifying integration with Google APIs. In addition a user always has the option to revoke access to an application at any time.

Our Google Sign-In feature is implemented using the `react-native-google-signin` library, in combination with Expo and Firebase. The library handles the authentication flow with Google. It allows users to sign in with their Google accounts and returns the necessary authentication tokens (such as the ID token) that can be used for further authentication with backend services. Unfortunately `react-native-google-signin` cannot be used in the standard Expo Go app because it requires custom native code. This applies to both the original and universal (one-tap) sign-in methods. To work around this, we use an Expo development build, which allows adding custom native modules. Development builds are the recommended approach for production apps when custom native code is needed. In order to link the google authentication to our firebase backend, after a user successfully signs in with Google, the ID token received from `react-native-google-signin` is passed to Firebase Authentication. Firebase uses this token to verify the user's identity and either sign them in or create a new user record. This links the Google account to our Firebase-authenticated user system.

Authentication flow summary:

1. The user taps "Sign in with Google."
2. `react-native-google-signin` starts the Google sign-in process and returns an ID token.
3. The ID token is sent to Firebase Authentication.
4. Firebase verifies the token and signs in (or registers) the user in our backend.

### 2.4.3. Facebook Sign-In

Facebook Sign-In manages the OAuth 2.0 flow and token lifecycle, simplifying integration with Facebook APIs. In addition a user always has the option to revoke access to an application at any time.

Our Facebook Sign-In feature is implemented using the `react-native-fbsdk-next` library, in combination with Expo and Firebase. The library handles the authentication flow with Facebook as the Google sign-in integration. It allows users to sign in with their Facebook accounts and returns the necessary authentication tokens (such as the ID token) that can be used for further authentication with backend services. In order to link the facebook authentication to our firebase backend, after a user successfully signs in with Facebook, the ID token received from `react-native-fbsdk-next` is passed to Firebase Authentication. Firebase uses this token to verify the user's identity and either sign them in or create a new user record. This links the Facebook account to our Firebase-authenticated user system. Authentication flow summary:

1. The user taps "Sign in with Facebook."

2. `react-native-fbsdk-next` initiates the Facebook sign-in process and returns an ID token.
3. The ID token is sent to Firebase Authentication.
4. Firebase verifies the token and signs in (or registers) the user in the backend.

#### 2.4.4. Maps

Integrating maps in mobile React applications is a common requirement for location-based features such as geolocation, routing, and displaying points of interest. There are several approaches available for adding maps to a React Native app. Some solutions use web-based maps rendered in a `WebView`, while others rely on native modules for better performance and integration.

Among the available options, `react-native-maps` remains the most widely used and robust solution. This library supports key features such as custom markers, polylines, region tracking, and user location display.

Although Expo currently displays a warning about using `react-native-maps`—primarily because it requires custom native code and can introduce compatibility challenges—it is still the most reliable choice. There are currently no well-established alternatives that offer the same level of stability, feature set, and community support. For these reasons, we continue to use `react-native-maps`, integrating it through a development build in Expo to ensure full access to the required native functionality.

We use maps in order to help users find shops that accept our app as payment method. In addition we also use the location service in combination with the maps API to allow owners to easily select their shop position when registering their shop.

#### 2.4.5. Google Cloud Functions

Google Cloud Functions serve as the backend logic layer of our application. These are serverless, event-driven functions that run in response to Firebase events such as Firestore writes. They are used to process backend logic dynamically, without the need for manual server deployment or scaling. In our implementation, Cloud Functions are triggered whenever a new contact is added, a new payment request is created, or a shop receives a transaction. Upon triggering, the function retrieves relevant data, formats a push notification payload, and forwards it to the Expo Push Notification service for delivery to the appropriate user.

#### 2.4.6. Expo Push Notifications

Expo Push Notifications are the messaging infrastructure used to deliver push notifications to users of our app. Instead of relying on Firebase Cloud Messaging (FCM) directly, we use Expo’s push service, which simplifies handling push tokens and cross-platform delivery. In this setup, each user registers their Expo push token (e.g., `ExponentPushToken[...]`) during login or registration, and these tokens are stored in Firestore.

When an event occurs, such as receiving a payment or adding a new contact, our backend functions send the notification payload to the Expo Push API, which then routes it to the appropriate device. This approach ensures that messages are delivered quickly and reliably across both iOS and Android, while allowing us to maintain a single, unified push system via Expo's infrastructure.

## 2.5. Dependencies

The application is developed using the React Native framework with Expo, allowing for rapid cross-platform development and streamlined access to native features. A set of carefully selected dependencies supports key functionalities such as authentication, real-time data, routing, maps, and push notifications. The following are the main dependencies that form the backbone of the app:

- **expo** — Provides a managed workflow for building React Native apps. It abstracts away native configurations, giving access to a rich ecosystem of APIs and services while simplifying development, testing, and deployment.
- **react-native** — The core framework for building mobile applications using JavaScript and React. It bridges JavaScript with native platform components, enabling native performance and a unified codebase for both iOS and Android.
- **react** — The JavaScript library used to create the user interface through component-based architecture. It powers the rendering logic and state management throughout the app.
- **expo-router** — Implements file-based routing, enabling the creation of navigable screens based on the directory structure. It simplifies the navigation model and integrates seamlessly with Expo and React Navigation.
- **@react-navigation/native** — Provides a complete navigation system, including stack, tab, and drawer navigators. It is responsible for managing the screen stack, transitions, and navigation state throughout the app.
- **@react-native-firebase/app** — Initializes Firebase services within the app. It is a foundational package required by other Firebase modules such as Firestore and Authentication.
- **@react-native-firebase/auth** — Integrates Firebase Authentication, supporting secure user sign-in flows, including email/password and federated identity providers like Google.
- **@react-native-firebase/firestore** — Enables real-time database functionality using Cloud Firestore. It supports structured, scalable data storage and live updates to the client app.
- **@react-native-google-signin/google-signin** — Adds Google authentication capabilities, allowing users to sign in using their Google account. It integrates with Firebase Auth for seamless backend handling.

- **expo-notifications** — Handles both remote (push) and local notifications. It manages permission prompts, message scheduling, and display, supporting FCM for delivery across Android and iOS.
- **react-native-maps** — Embeds maps directly in the application using native map views. Despite Expo’s current caution against using this package in managed workflows, it remains the most robust and widely used solution for map functionality.
- **expo-camera** — Provides access to the device camera for photo and video capture. It supports use cases like scanning, user media uploads, and QR code capture.
- **expo-location** — Enables access to location services, allowing the app to fetch geographic coordinates and track user movement when permissions are granted.
- **expo-device** — Supplies runtime information about the user’s device, such as model name, operating system, and device ID, which can be useful for debugging and analytics.
- **expo-document-picker** — Lets users select documents from their device storage, supporting workflows like uploading PDFs, spreadsheets, or text files.
- **react-native-svg** and **react-native-qrcode-svg** — These libraries are used for rendering scalable vector graphics and generating QR codes dynamically within the app interface.
- **expo-screen-orientation** — Allows the app to lock or respond to changes in screen orientation (portrait/landscape), used in conjunction with gyroscope data to enhance map and navigation experiences.
- **react-native-fbsdk-next** — Provides support for Facebook login integration. Though not used in the core flow, it enables optional social authentication via Facebook, enhancing sign-in flexibility for users.
- **expo-linking** — Used to open external applications from within the app, such as launching Google Maps with a specific location. This enables seamless transitions between the app and other installed services on the device.
- **expo-constants** — Provides access to system constants such as the app manifest, device ID, and other environment-specific values. In this app, it is used primarily to support push notifications by retrieving necessary identifiers for token registration.
- **expo-font**, **expo-splash-screen** — Utility libraries that support font loading and splash screen configuration, respectively.

## 2.6. Component Architecture

This application adopts a component-based architecture to structure the user interface and manage interaction logic. Components are the fundamental building blocks of the app, each encapsulating its own rendering, behavior, and styling. Our approach emphasizes modularity, reusability, and clarity, making it easier to develop, maintain, and evolve the application over time.

This section details how components are organized, the conventions we follow when creating new components, and how they integrate to form a cohesive user experience. By adhering to this architecture, we ensure consistency, scalability, and efficient team collaboration across the codebase.

### **2.6.1. Layout Components**

Layout components are designed to provide structural organization. They act as containers for other components, handling positioning, scrolling, spacing, and general layout management. Examples include:

- `FormLayout.tsx`
- `SplitScrollableLayout.tsx`

These components are primarily concerned with arranging and nesting child components and rarely contain business logic.

### **2.6.2. Section Components**

Section components are smaller, self-contained parts of a page that group a specific piece of functionality or visual content. Unlike layout components, section components do not accept dynamic children and have a fixed structure. Examples include:

- `ChatSection.tsx`
- `InfoCardSection.tsx`

Sections are used to build larger pages by combining multiple non-dynamic visual or functional units.

### **2.6.3. Listable Components**

Listable components represent repeated list items within structured lists. They are designed to be used in contexts where many instances of the same kind of data are displayed uniformly. Examples include:

- `ContactListable.tsx`
- `MessageListable.tsx`
- `TransactionListable.tsx`

Each listable is optimized for consistent rendering and efficient updates when datasets change.

### **2.6.4. Modal Components**

Modal components represent popup overlays or dialogs used throughout the application to capture user input, show details, or perform contextual actions. Examples include:

- `AppModal.tsx`
- `ContactRequestModal.tsx`
- `MessageModal.tsx`
- `PaymentRequestModal.tsx`
- `PermissionRequestModal.tsx`
- `TransactionDetailModal.tsx`

Due to the nature of the app, modals play a significant role in user interactions, ensuring key workflows (like payments or permissions) are seamless and focused.

### 2.6.5. Input Components

Input components provide consistent, reusable input fields across the application, ensuring a unified design and validation behavior. Examples include:

- `InputDocuments.tsx`
- `InputLocation.tsx`
- `InputText.tsx`

Inputs are designed to be easily integrated into forms and modals, minimizing redundancy and promoting consistent UX patterns.

### 2.6.6. Base Components

Base components are the smallest and most reusable building blocks in the application. They provide themed primitives like buttons, labels, and error text, which are then composed into larger components. Examples include:

- `AddContactButton.tsx`
- `BaseError.tsx`
- `BaseLabel.tsx`
- `BaseText.tsx`
- `LabelRow.tsx`
- `LabelValueRow.tsx`

These components are kept as lightweight and generic as possible to maximize reuse and maintain design consistency.

## 2.7. Sequence Diagrams

The following is a selection of the most relevant sequence diagrams.

### 2.7.1. Sign Up with Google

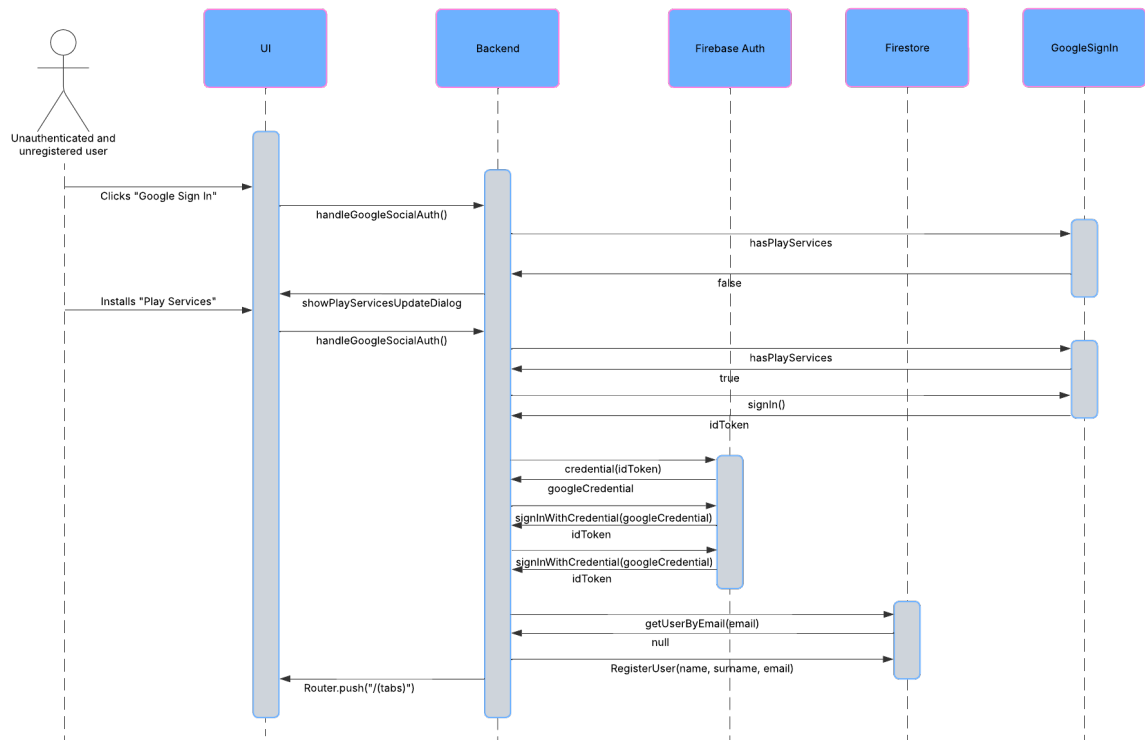
The following diagram shows the process of sign up of an unauthenticated user. The flow starts when the user initiates a sign-in action from the UI. The application integrates Google Sign-In to authenticate the user and Firebase services to manage authentication and store user data.

Once the user triggers the Google sign-in process, the ‘GoogleSignIn’ service handles the authentication UI and provides an ID token. This token is sent to Firebase Authentication to validate and issue a Firebase session. Upon successful authentication, the backend receives the Firebase user credentials and proceeds to verify whether the user is new or already exists in Firestore.

If the user does not exist in the Firestore database, a new document is created with the relevant user data. The backend may perform additional setup or validations before responding to the frontend with a completed authentication state and user profile. The UI is then updated to reflect the signed-in state, allowing the user to proceed within the app.

Even though this process involves multiple asynchronous interactions and backend verifications, the experience for the user remains seamless and straightforward — reduced to a simple tap on the "Sign in with Google" button followed by access to the application.

It is important to note that the flow shown in the diagram represents a worst-case scenario in which the user does not even have Google Play Services installed on the device. In most cases, the process is much faster and lighter. Additionally, sign-up and sign-in flows are unified in this implementation — the system handles both transparently — and therefore no separate diagram is provided for the sign-in flow.



**Figure 2.1: Worst-case Google Sign-Up Flow integrating UI, Backend, Google Sign-In, Firebase Auth, and Firestore.**

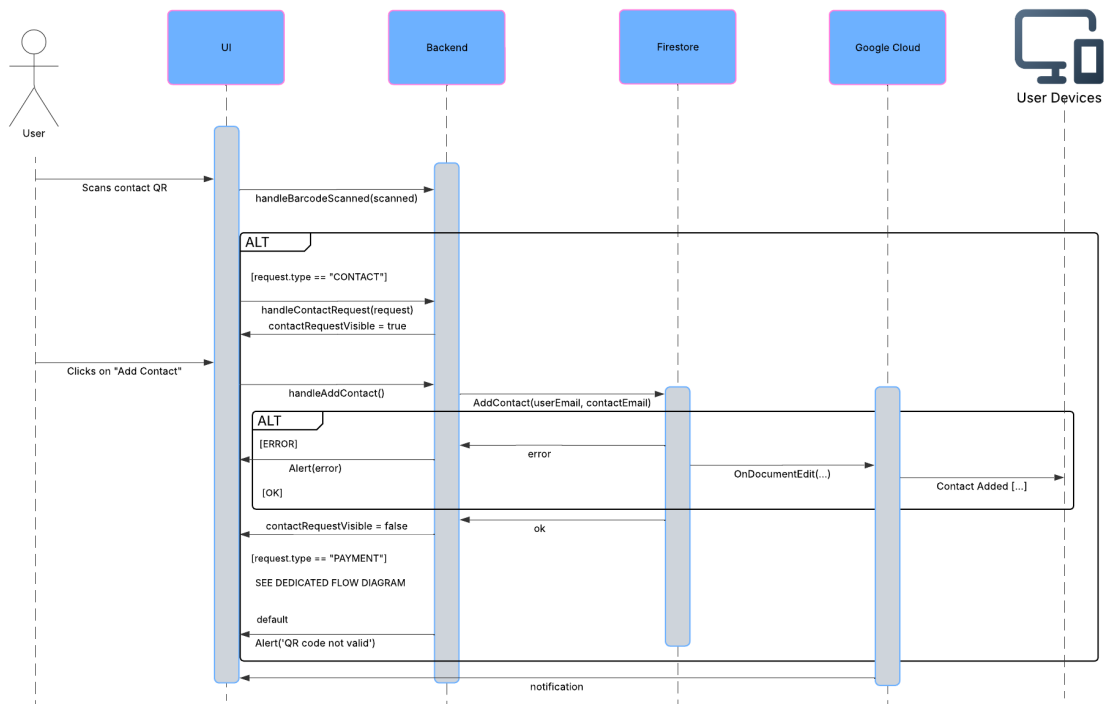
## 2.7.2. Add Contact

The following diagram illustrates the process a user goes through to add a new contact by scanning a QR code. This flow engages multiple layers of the app architecture, including the UI, the backend logic, and Firestore integration.

The process begins when a user scans a QR code that encodes a contact request. The UI layer receives this scan event and calls a handler that interprets the request. If the type of request is identified as **CONTACT**, the UI makes a dedicated contact request visible to the user, who can then choose to proceed.

Once the user confirms the addition by clicking on "Add Contact", a backend handler attempts to persist the contact association in Firestore. The shown method (AddContact) is an high level abstraction of the operations made on the actual DB. It's also important to note that the contact is added to both users: the DB operation is symmetrical.

In the event of an error during this operation (e.g. contact not found, contact already added), the user is notified through a visual alert. If successful, the contact request UI is dismissed. The diagram also shows fallback conditions: if the QR code corresponds to a payment, a different flow is invoked (described in its own section); otherwise, the user is alerted that the QR code is invalid.



**Figure 2.2: Contact addition flow initiated by QR code scanning and confirmed by user interaction.**

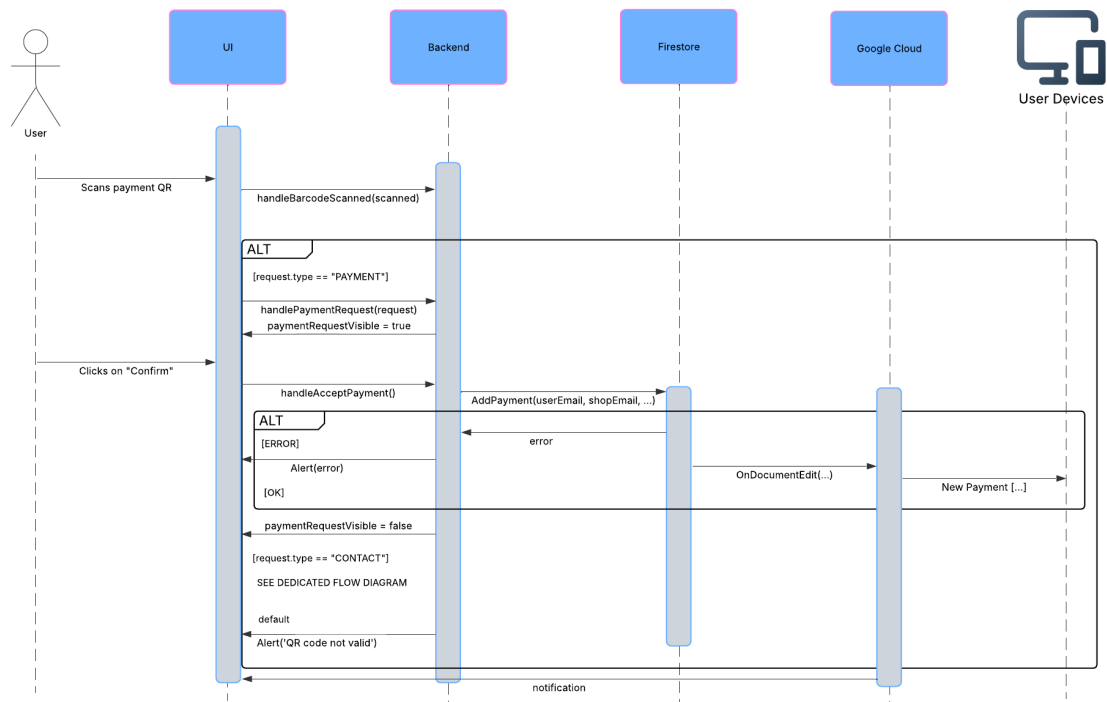
### 2.7.3. Pay using QR

The following diagram illustrates the process a user follows to initiate and confirm a payment by scanning a QR code. This flow engages multiple components of the system architecture, including the user interface (UI), backend logic, and Firestore database operations.

The process begins when the user scans a QR code that encodes a payment request. The UI receives the scan event and triggers a handler function that parses the request. If the request type is identified as **PAYMENT**, the payment request becomes visible to the user.

After reviewing the request, the user confirms the transaction by clicking on the "Confirm" button. This action invokes a function in the UI that calls the backend to process the payment. The backend handles this by making a call to a method called **AddPayment**, which is responsible for persisting the payment record in Firestore. This abstraction represents the database operation that stores relevant details such as the user and shop emails.

If an error occurs during this process (e.g. invalid data), the error is identified and a descriptive message is propagated back to the UI, where it is surfaced to the user through an alert dialog. On a successful transaction, the payment request UI is dismissed. Finally if the QR code does not match any known request type, the user is notified with an alert stating that the code is not valid.



**Figure 2.3: Payment flow initiated by QR code scanning and confirmed by user interaction.**

#### 2.7.4. Add Shop Flow

The following diagram illustrates the process a user follows to create a new shop within the application. This interaction involves the user interface (UI), backend services, and Firestore database operations.

The process begins when the user initiates the action by clicking the "Add Shop" button. This interaction triggers a function on the UI, which in turn calls the backend to handle the logic for creating the shop and registering payment data. On the backend, the data inserted in the form by the user is processed and a function called `AddPayment` is used to update the DB (Firestore). This function takes several parameters, including the user's email and the shop email, and is responsible for persisting the payment record associated with the new shop in Firestore, the details of the operations on the DB are not shown for simplicity. Upon completion of this database operation, the backend returns either a success message or an error. This outcome is handled via an alternate flow: If an error occurs (e.g., network failure, invalid input), it is captured and returned to the UI. The UI then surfaces the issue to the user via an alert dialog displaying the error message. If the operation succeeds, a confirmation alert is shown. Once the user acknowledges the success message by clicking "OK", the application navigates to the shop overview screen.

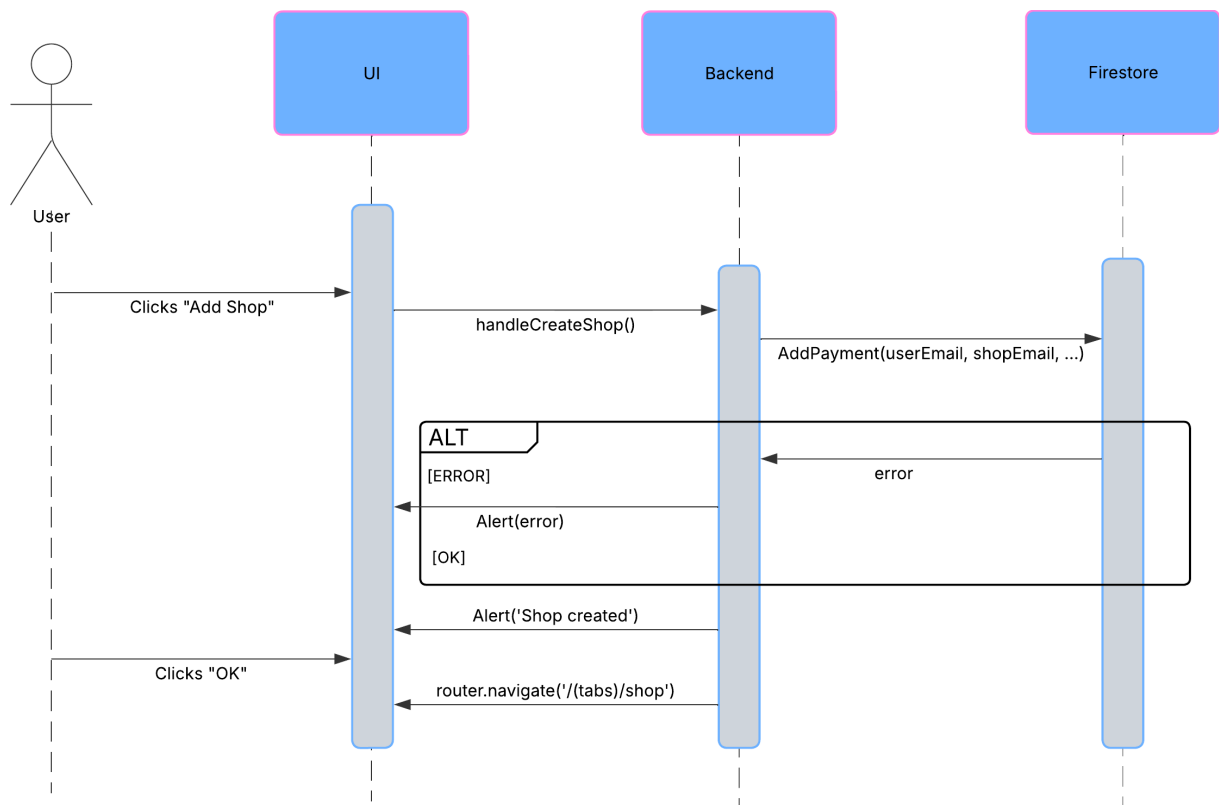
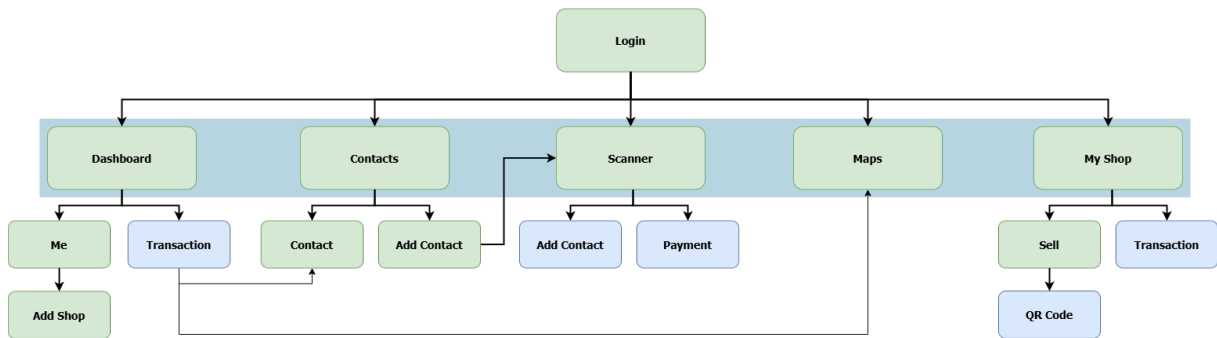


Figure 2.4: Shop creation flow triggered by user interaction and confirmed by backend and database updates.

# 3 | User Interface (UI)

## 3.1. Navigation

Before delving in how the user interface has been designed it is better to look at the application from a navigational point of view.

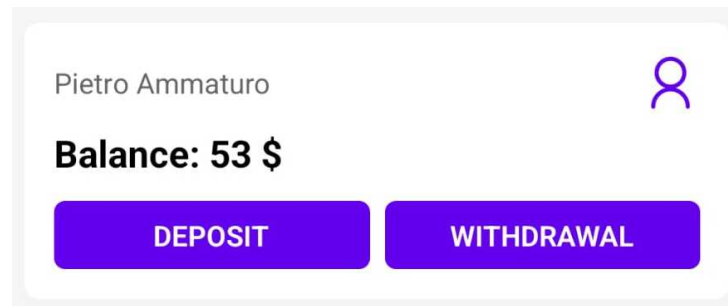


**Figure 3.1: High level navigation view.**

We can observe how, after the first login screen, the user can navigate seamlessly between several different screens, such as Dashboard, Contacts, Scanner, Maps and My Shop (optional). This navigation is handled by a tab navigation which is an approach similar to several other similar applications. While inside each of these main application sections users can go to deeper pages (in green) or modals (in blue) using a stack navigation approach. In some cases cross section navigation is allowed, such as when going directly from the transaction modal to the contact screen.

## 3.2. UI Design Choices

The UI Design has been studied to be as intuitive and clear as possible, with special focus on ensuring consistency among interactions and a clear differentiation between interactable and non-interactable elements. All interactable elements not located in the bottom or top tabs are highlighted in a distinct color.



**Figure 3.2: Example of in-section interactable elements and buttons**

The only notable exception is list-based elements such as transactions or contacts.

|                           |               |
|---------------------------|---------------|
| <b>Cash movement</b>      |               |
| jek8888888@gmail.com      | <b>-50 \$</b> |
| Withdrawal                |               |
| <b>Andres Bonilla</b>     |               |
| bonillaq.andres@gmail.com | <b>-12 \$</b> |

**Figure 3.3: Example of list-based interactables**

A distinction is also made between intra-section links (which are represented by buttons, similar to other interactable elements) and cross-section links, which are styled more like traditional hyperlinks.

|                                             |
|---------------------------------------------|
| <b>Name:</b> <a href="#">Andres Bonilla</a> |
| <b>Email:</b> bonillaq.andres@gmail.com     |
| <b>Amount:</b> -12 \$                       |
| <b>Description:</b>                         |

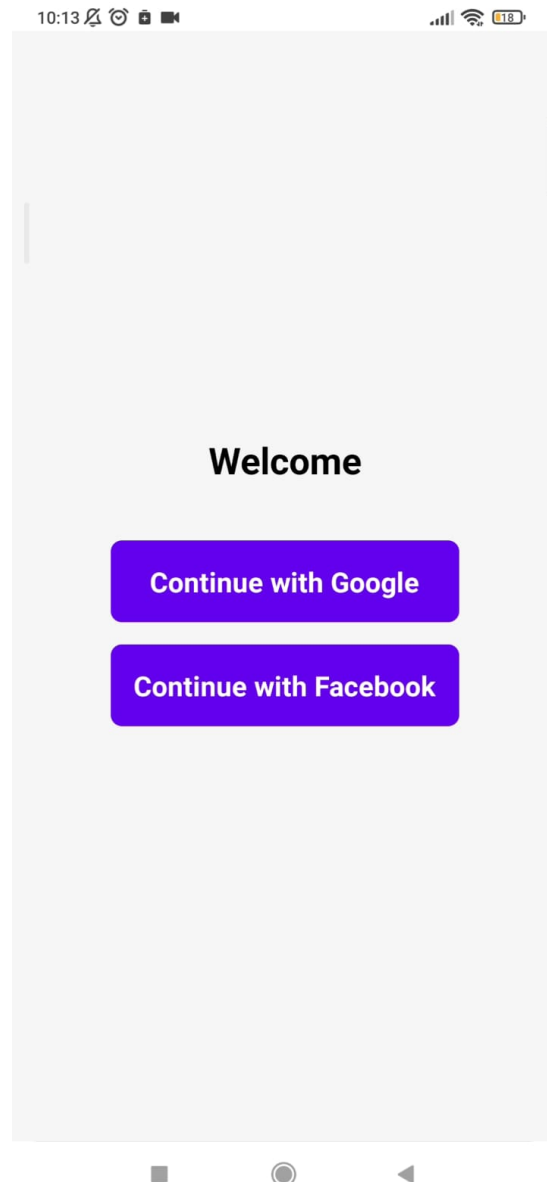
**Figure 3.4: Example of cross-section**

## **3.3. Smartphone UI**

### **3.3.1. Login and Registration**

The following image covers the login and registration screen. The authentication process has been outsourced to trusted entities, such as Google and Facebook. This allows for a simple and clean welcome screen with just two basic buttons, to let users choose the

provider they prefer.



### 3.3.2. Dashboard

The following images depict the Dashboard screen, as we can see it is composed from two main panels. The first panel contains general information about the user, such as name and balance, it also gives access to several functionalities such as deposit and withdrawal (mocked in the context of this project) and exposes a link to the personal user profile in the form of an icon in the top right corner, which can be clicked to go to the user's profile screen. The second panel contains a list of the user's most recent transactions, initially only a limited number will be loaded to minimize the request's delay, upon user scroll more transactions will be loaded. It is possible to click on the individual transaction to see its details, here a cross section link is shown who will redirect the user either to the chat page with the contact who made the transaction or to the maps page in case the

transaction was made in a shop.

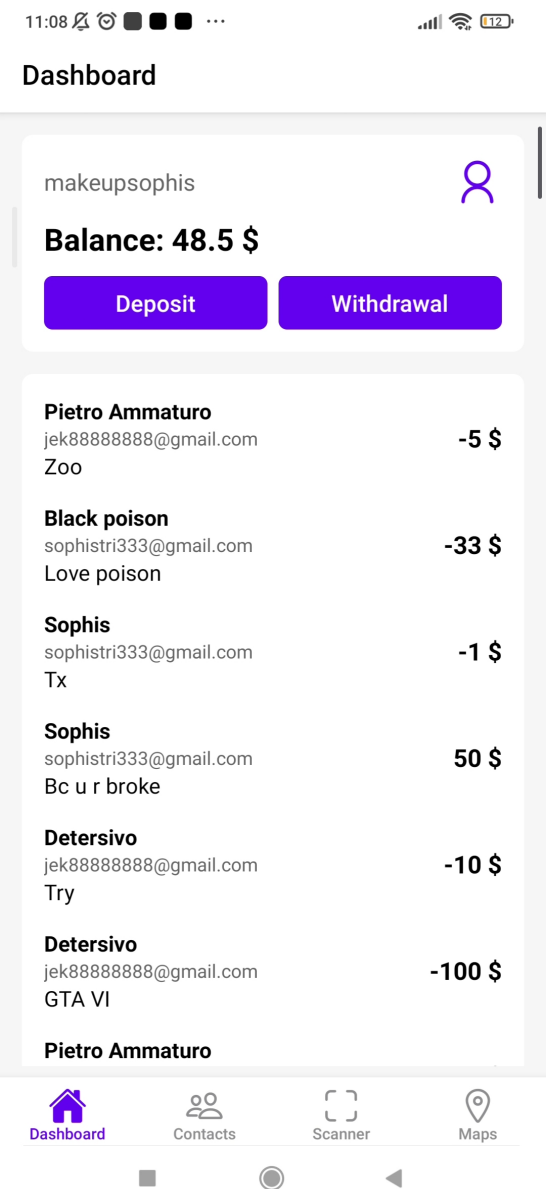


Figure 3.5: Dashboard Page.

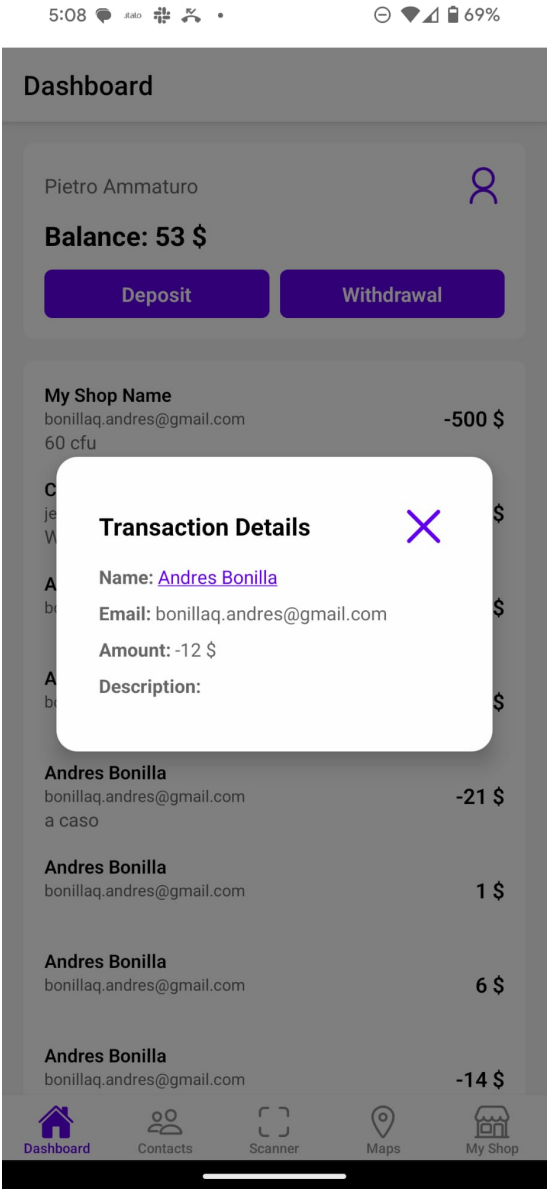


Figure 3.6: Dashboard Page with transaction details.

### 3.3.3. Me

The following images depict the Me screen, where the user is able to see the email he has created the account with as well as a scannable QR code that can be used from nearby users to add the user as contact. In the second image we can also see an add shop button which will redirect to the dedicated screen.

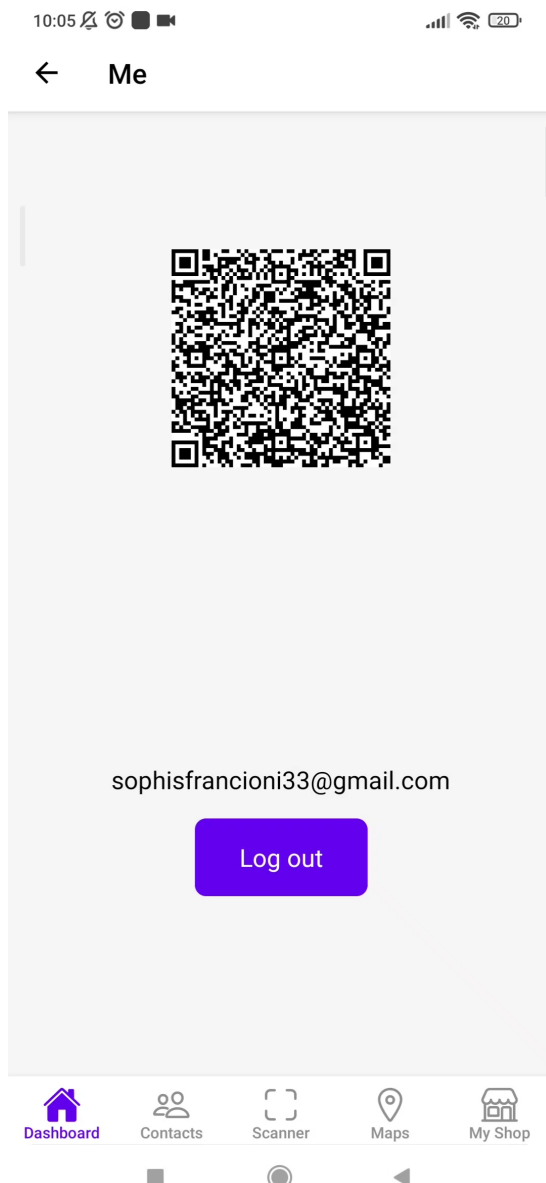


Figure 3.7: Me Page.

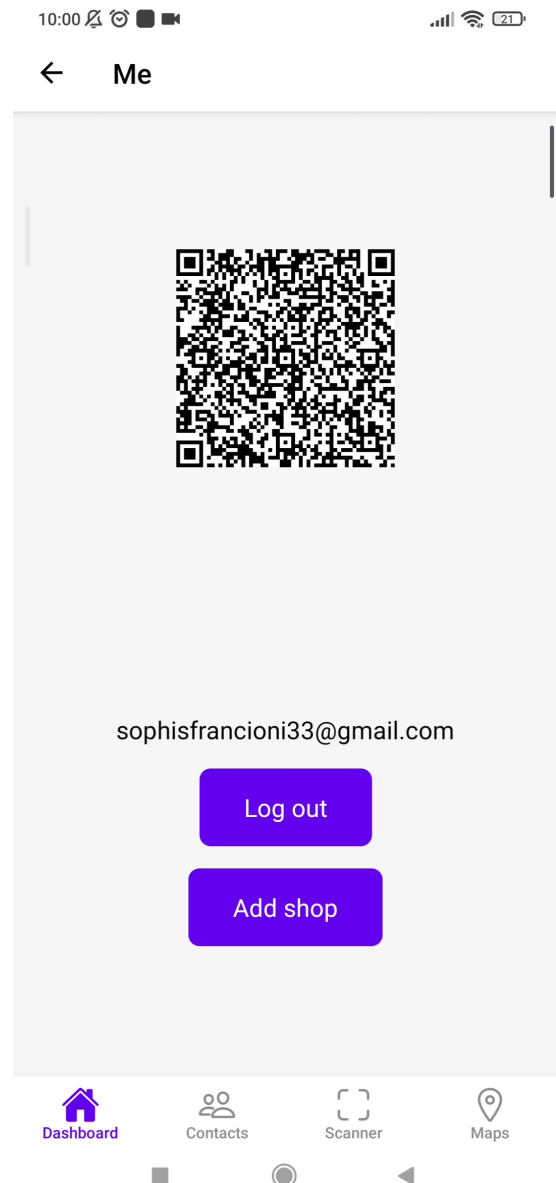


Figure 3.8: Me page with add shop.

### 3.3.4. Add Shop

In the following image we can see the add shop screen. Here a user who owns a shop can insert relevant information in order to add it to the shops who support the application as payment method. Notably name and description of the shop, as well as the location which can be easily selected on the map. The map interaction works by dragging the map around while the selected point remains fixed in the center of the map. This mechanism has proven successful in several commercial apps such as Deliveroo. While the filling of all form fields is checked from the application, the submitted contents are not verified and therefore the acceptance mechanism for new shops has been mocked as "always accepted".

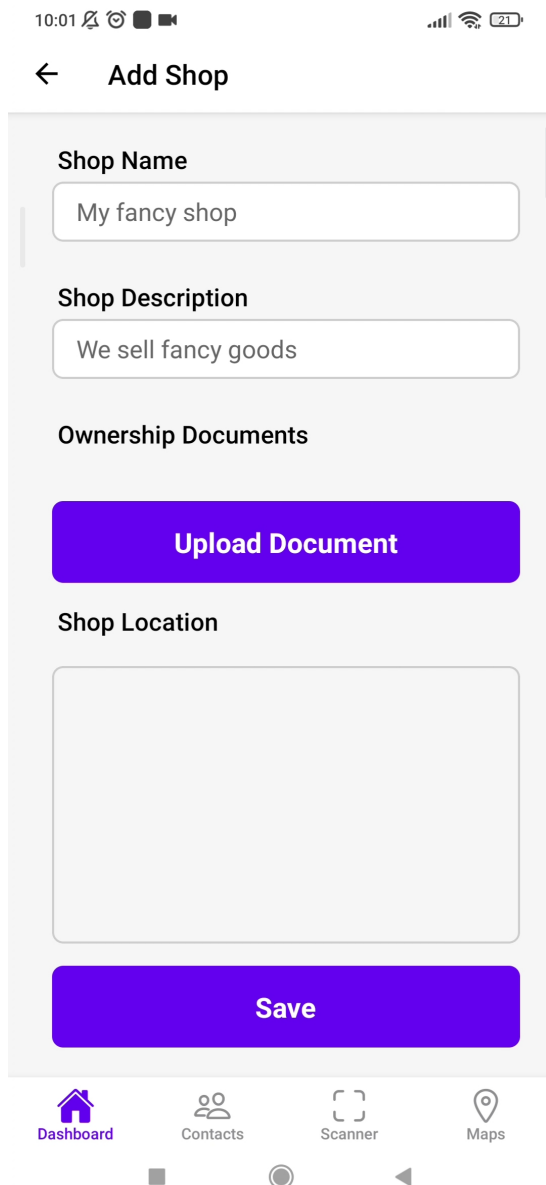


Figure 3.9: Add Shop.

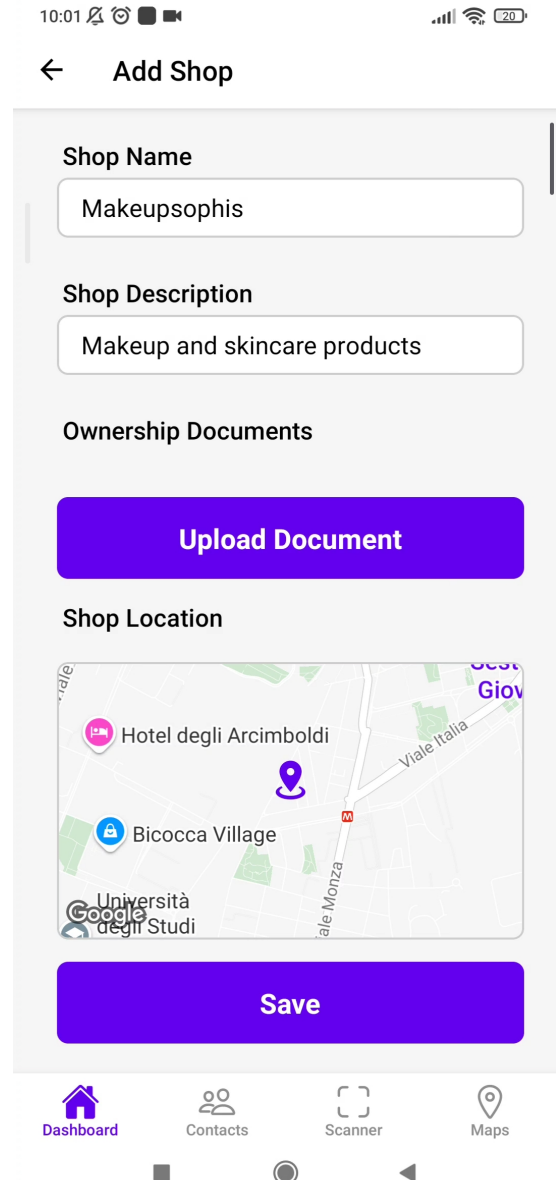
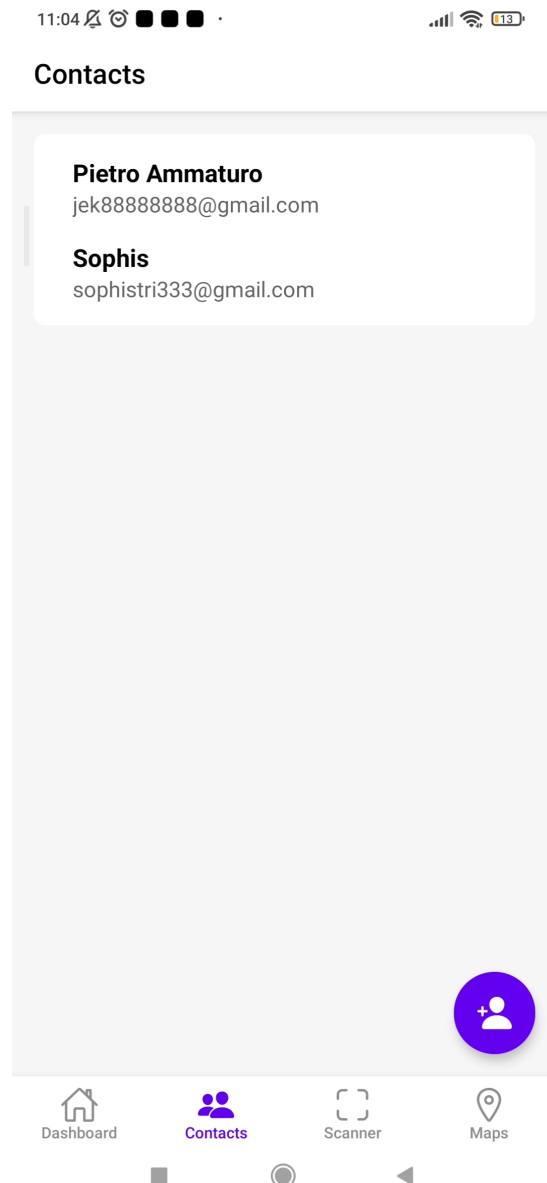


Figure 3.10: Add Shop (compiled, with visible map).

### 3.3.5. Contacts

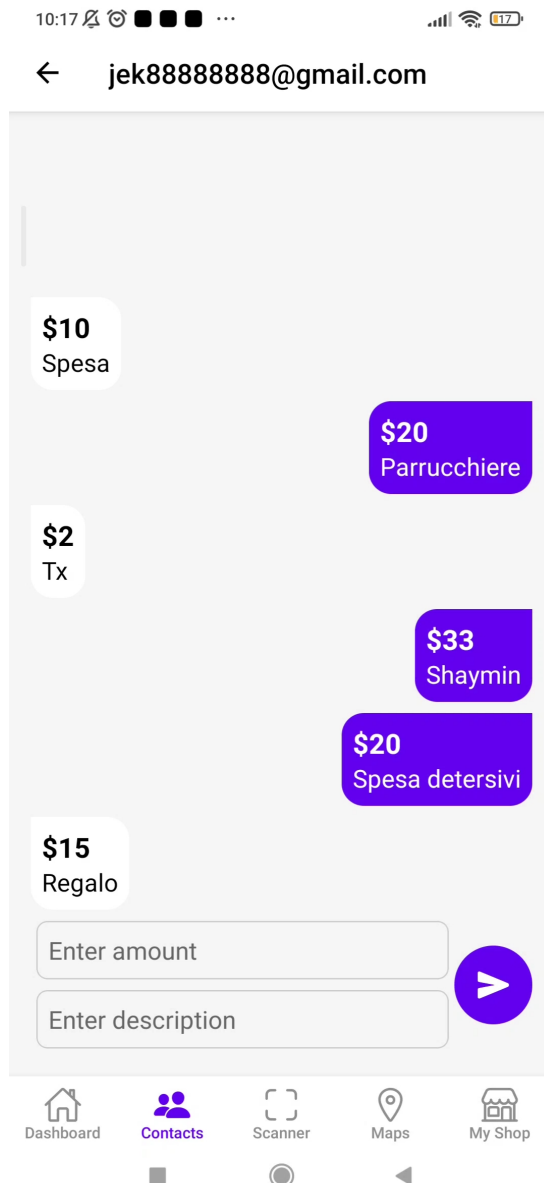
This is the contacts page, as we can see its contents are minimal: there is just a list of contacts of which e-mail and name are shown and a button to the bottom right which redirects to the add contact page. Here too, like in the transaction's case, a restricted number of contacts is loaded, to avoid excessive delays. Upon page scroll more contacts are loaded. Clicking on any of the conacts will redirect to the individual contact page wich will be a chat containing the transactions of the two users.



**Figure 3.11: Contacts Page.**

### **3.3.6. Chat**

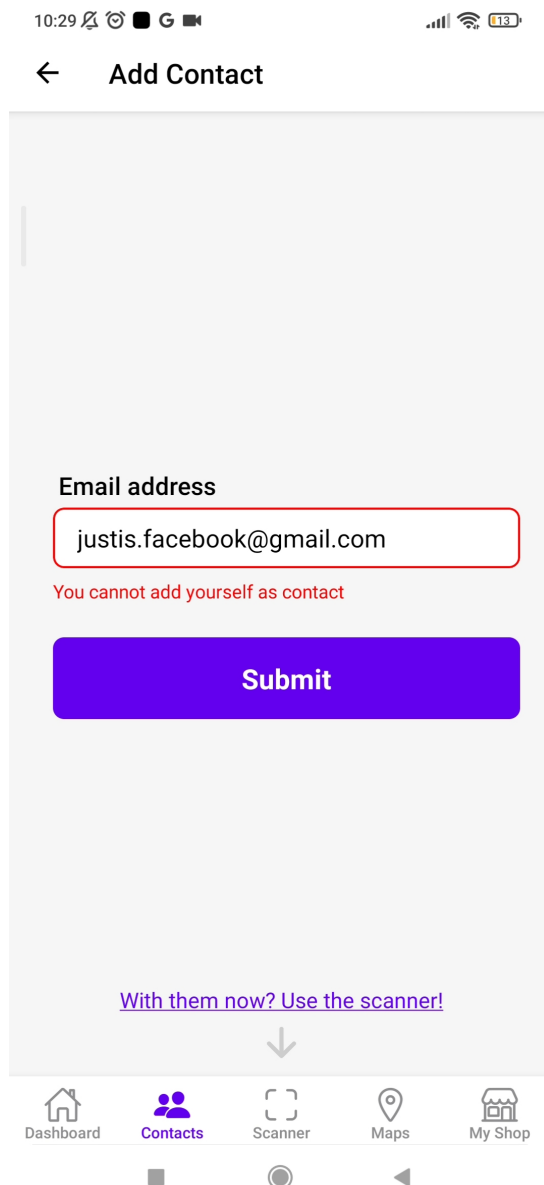
This is the chat page, contrary to what the name suggests the main goal of this page is to allow users to send money to each other in an intuitive manner. This design has been proven successful by several banking apps such as Revolut. In order to send money users just need to input amount and description for the transaction and click the send button to the right of the form.



**Figure 3.12: Chat Page.**

### **Add Contact**

Here is the add contacts page, a user can enter the email of another user to add him as contact. This is not the only way to add contacts though, as the cross section link on the bottom of the page suggests the scanner functionality of the application can be used to add a contact.



**Figure 3.13: Add Contact Page.**

### **3.3.7. Scanner**

The scanner page consists in a fullscreen camera view, with situational modals appearing according to the scanned QR code.

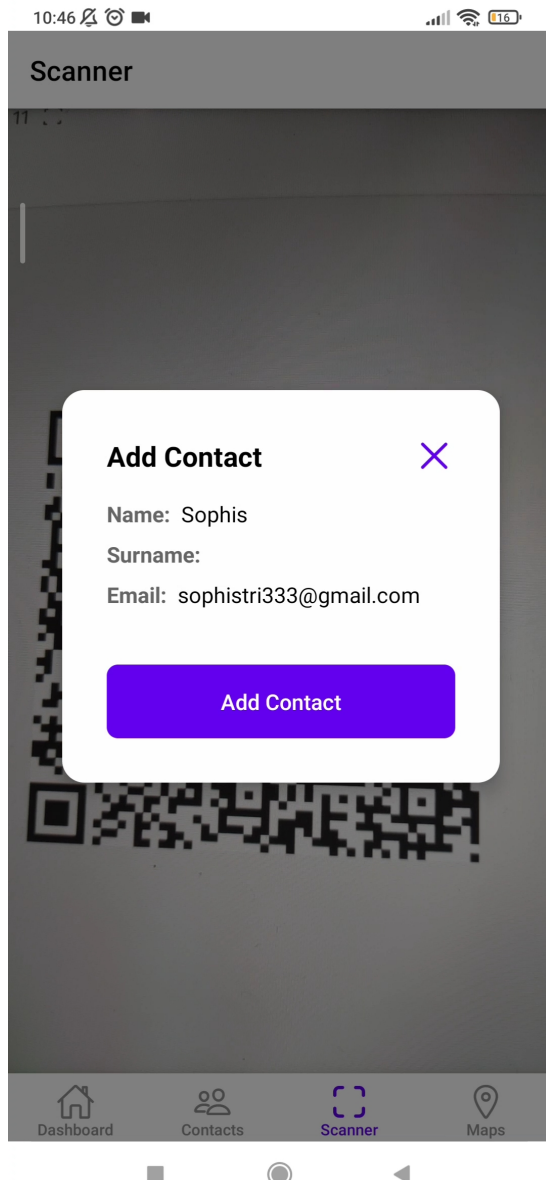


Figure 3.14: Scanner with add contact modal.

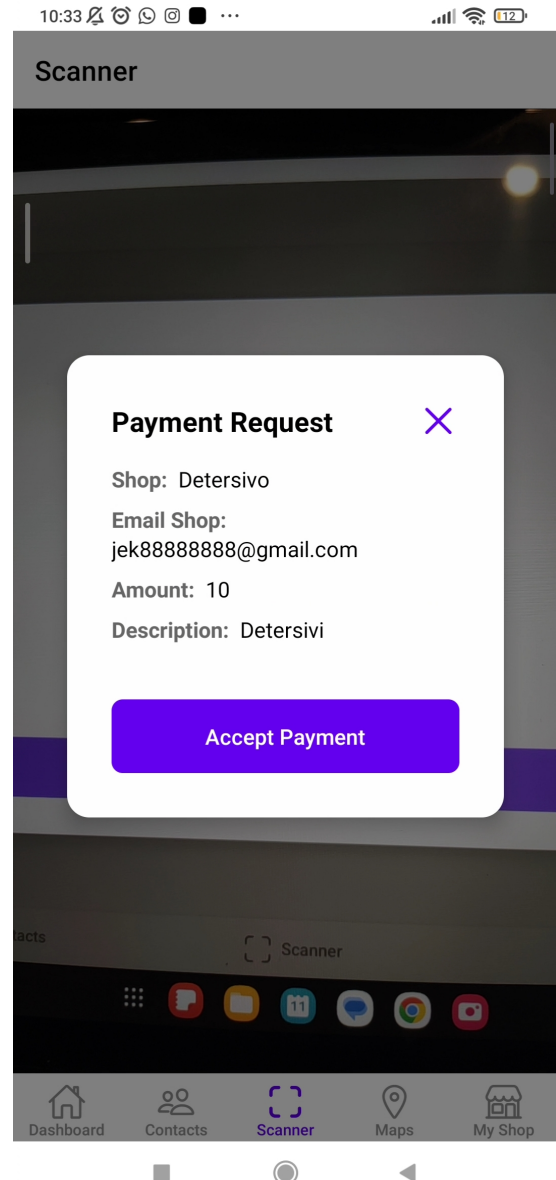
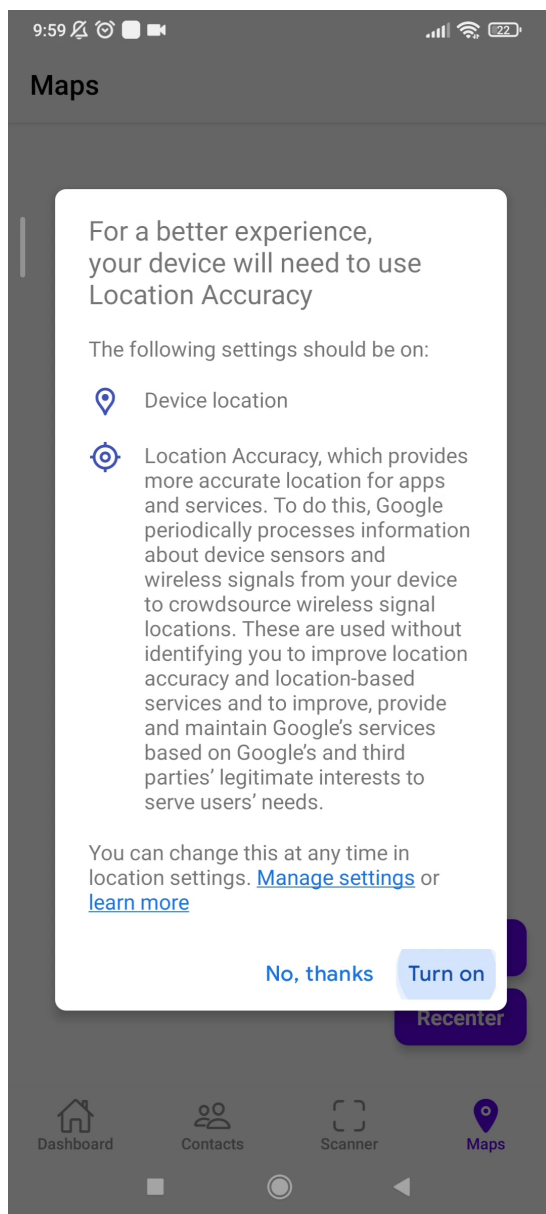


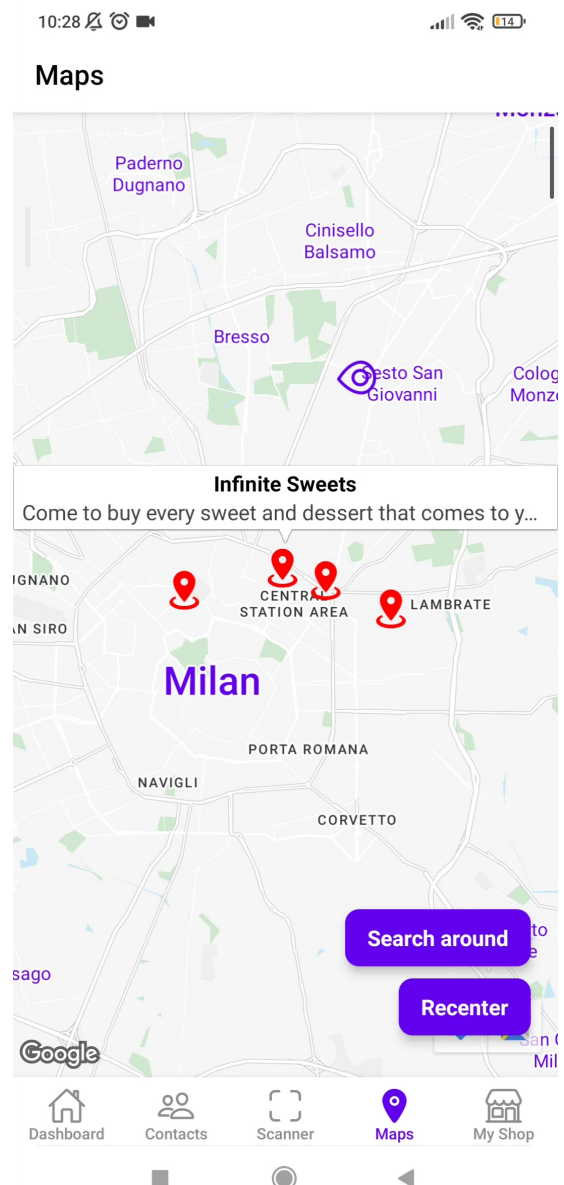
Figure 3.15: Scanner with pay modal.

### 3.3.8. Maps

Here we can see the Maps functionality of the application, this screen has two main goals. The first clear goal is to show to users in an intuitive way which shops support the application as payment method, this can easily be done by using the search around functionality. The other goal is to allow the user to identify in which shops his past transactions have happened: as we noted in the dashboard page, upon clicking a transaction and opening its details it is possible to click on the shop to go to the specific location on the map where the shop is located.



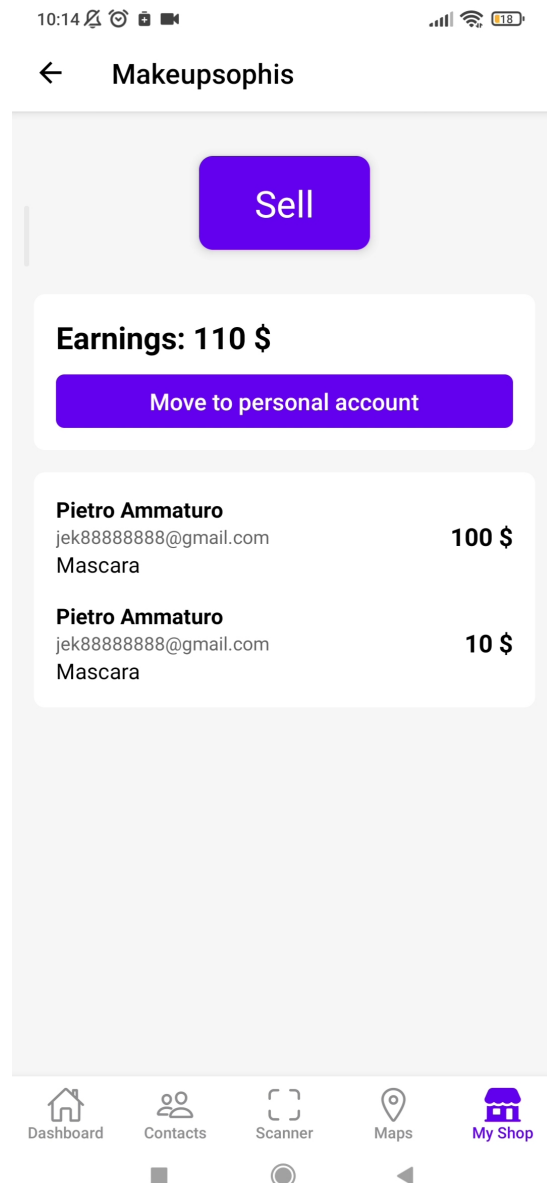
**Figure 3.16: Maps with permissions.**



**Figure 3.17: Maps.**

## Shop

The final section of the application is the My Shop section, here the main screen allows the user to: withdraw funds from their shop, see the transactions of their shop and more importantly sell, which can be done from the dedicated page which opens up after clicking the sell button.



**Figure 3.18: My Shop Page.**

## Sell

The sell page allows the user to select an amount and a description for the transaction and to generate a QR code which can then be scanned from the user to confirm the payment. This approach is drastically different from the one used from several e-pay applications such as "satipay" or "bonus cultura" where the buyer is the one who must insert the correct amount. In our case the approach is more similar to traditional pos based payments, where it is the seller who inserts amount and description, while the user accepts the payment.

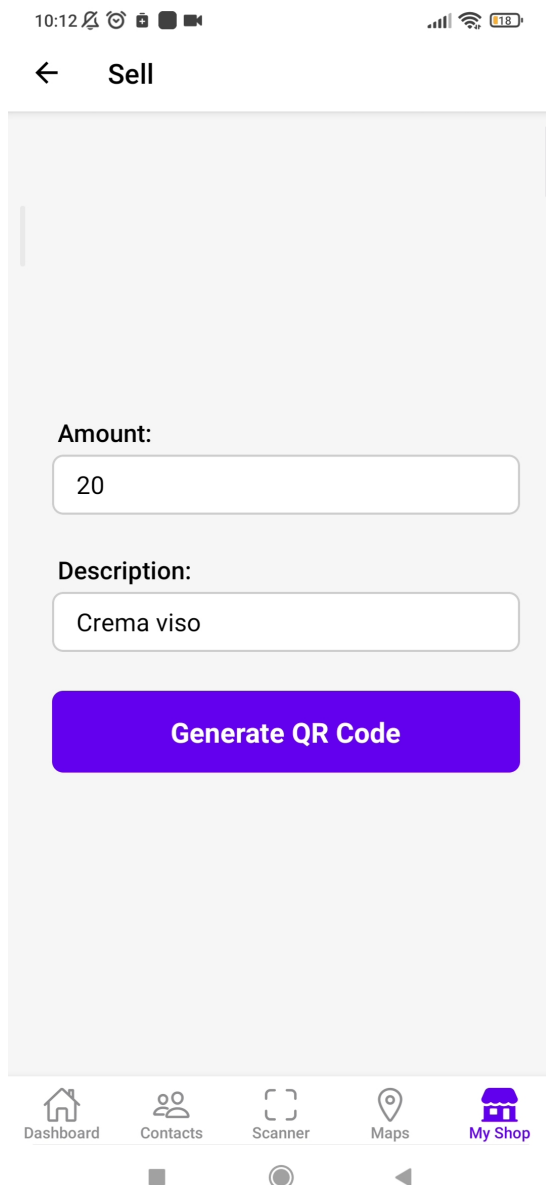


Figure 3.19: Sell.

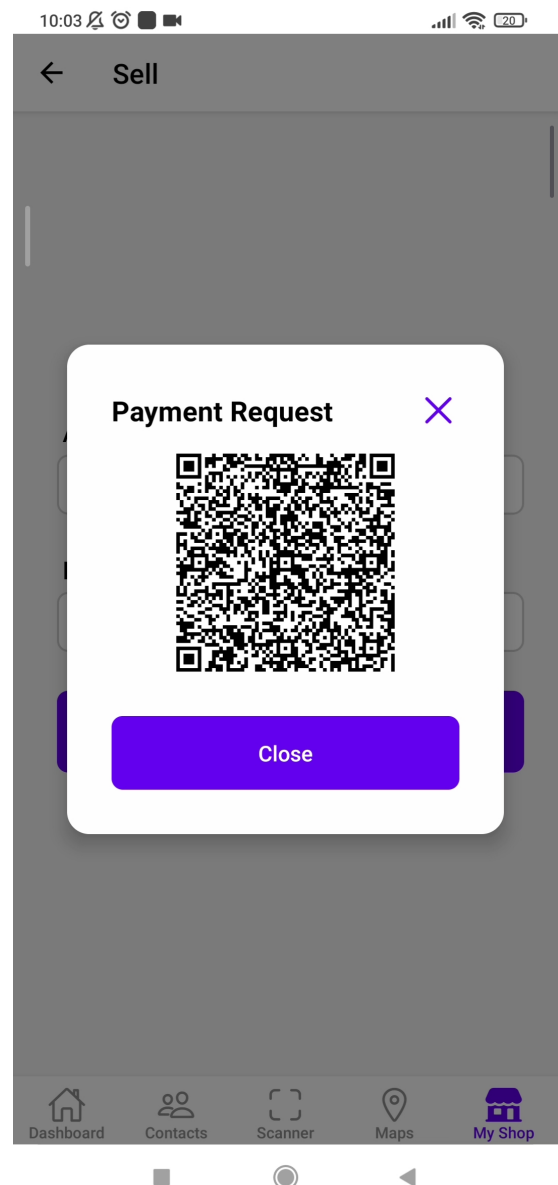


Figure 3.20: Sell with modal.

### 3.4. Tablet UI

In the following section we take a look at how the main screens change and adapt to bigger devices, such as Tablets. It is important to note that this layout changes are only applied in landscape mode, while the tablet UI remains the same as the smartphone's in portrait mode, for design reasons. As a positive side effect of the Tablet UI, the application's layout is responsive to device orientation, even if it is mainly intended to be used in portrait mode on smaller devices. We will not go through all the screens, as we did previously. We will rather focus on the most relevant ones and how they are changed.

### 3.4.1. Dashboard

As we can see, the Dashboard screen has undergone a mayor change: while the two main blocks were previously vertically aligned, now they are side by side, with the list of transactions (to the right) being the only scrollable part of the screen.

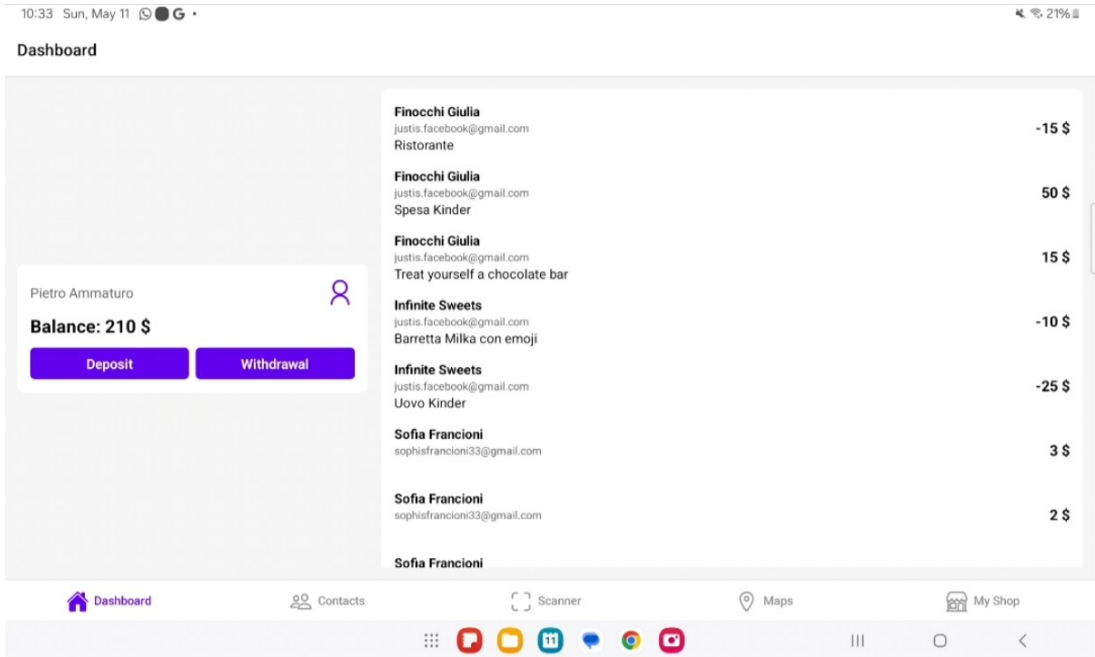


Figure 3.21: Dashboard landscape screen.

### 3.4.2. Shop

In a similar fashion to the Dashboard also the Shop screen follows the same structure, with the sell and move to personal account buttons being on the left part of the screen and the list of transactions to the right.

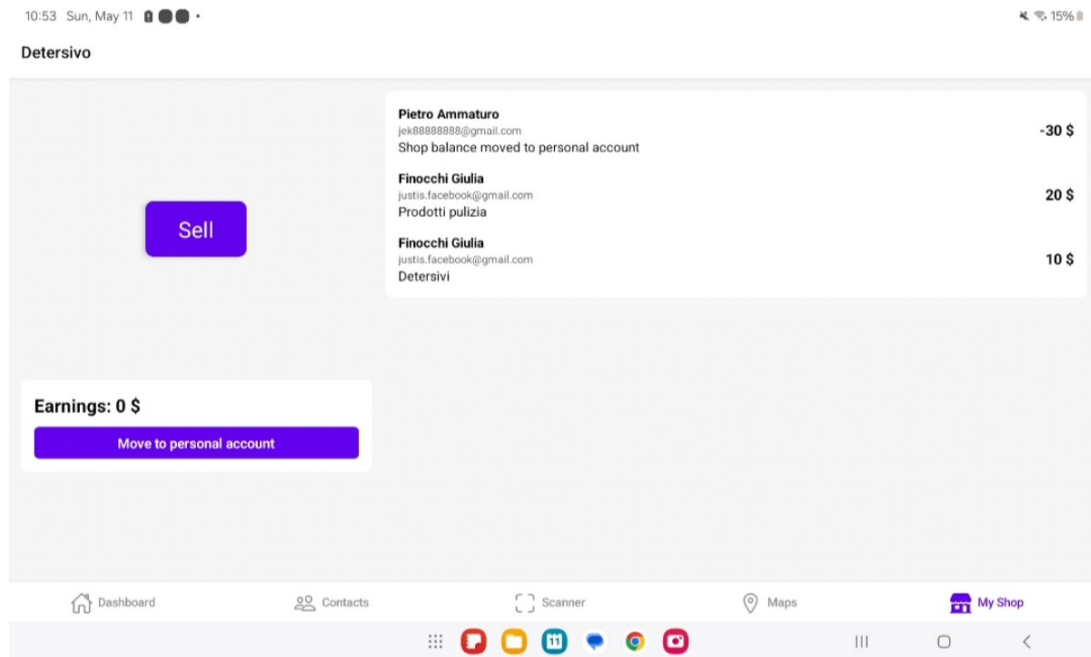


Figure 3.22: My Shop landscape screen.

### 3.4.3. Contacts & Chat

The contacts page is the one who has undergone the most relevant change: now the list of contacts can be seen to the left side of the page while individual messages are positioned to the right. This adaptation is not merely visual, as the navigation of the application was also adapted to integrate with the new design. In fact upon switching from landscape to portrait the layout will change and the user will be redirected to the selected chat page (if any). If switching back from portrait to landscape, the user will not only be redirected to the contacts page, but the individual chat screen will also be visible.

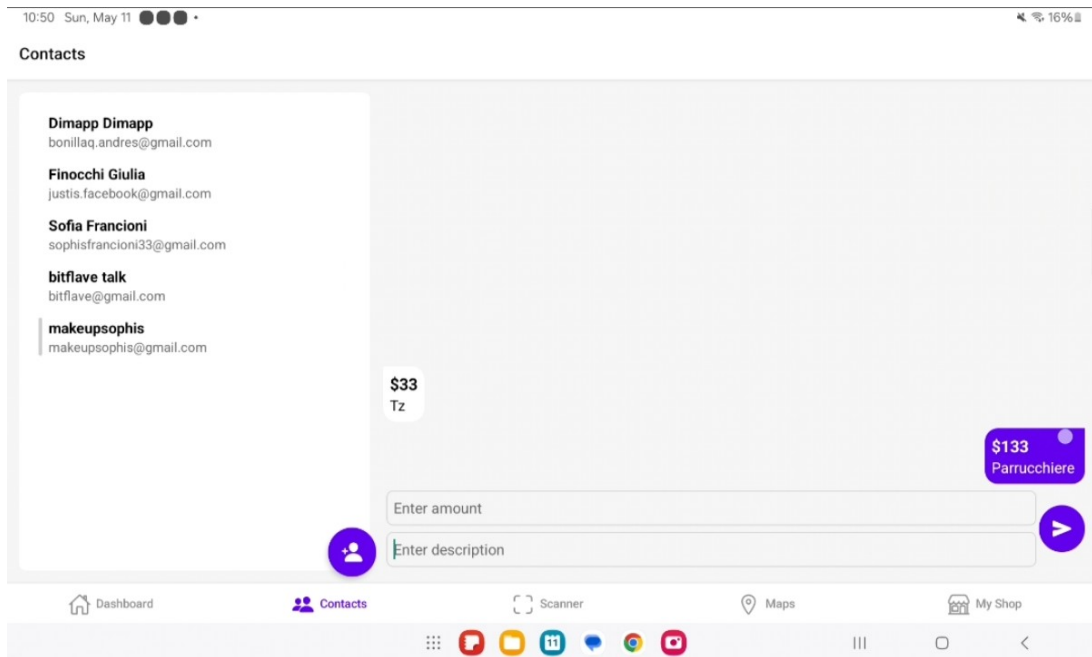


Figure 3.23: Contacts & Chat landscape screen.

### 3.4.4. My Shop

The Add Shop screen too has been adapted to the different layout, giving the user more space to select the shop position on the map.

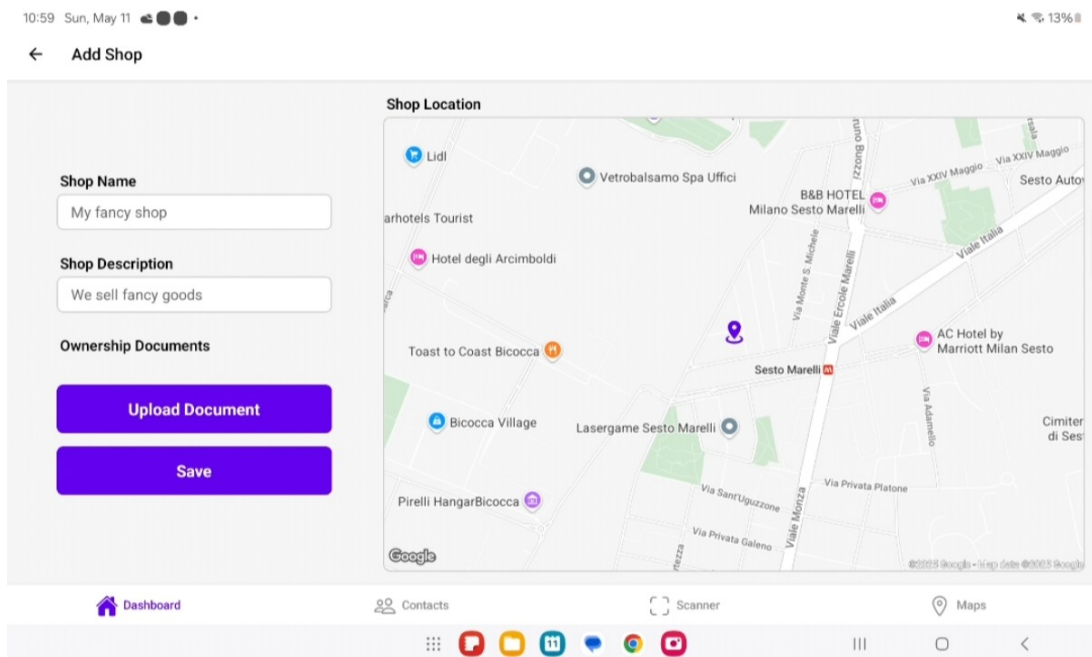


Figure 3.24: Add Shop landscape screen.

### 3.4.5. Me

The Me screen too has been adapted to the different layout, properly separating the QR and the email elements.

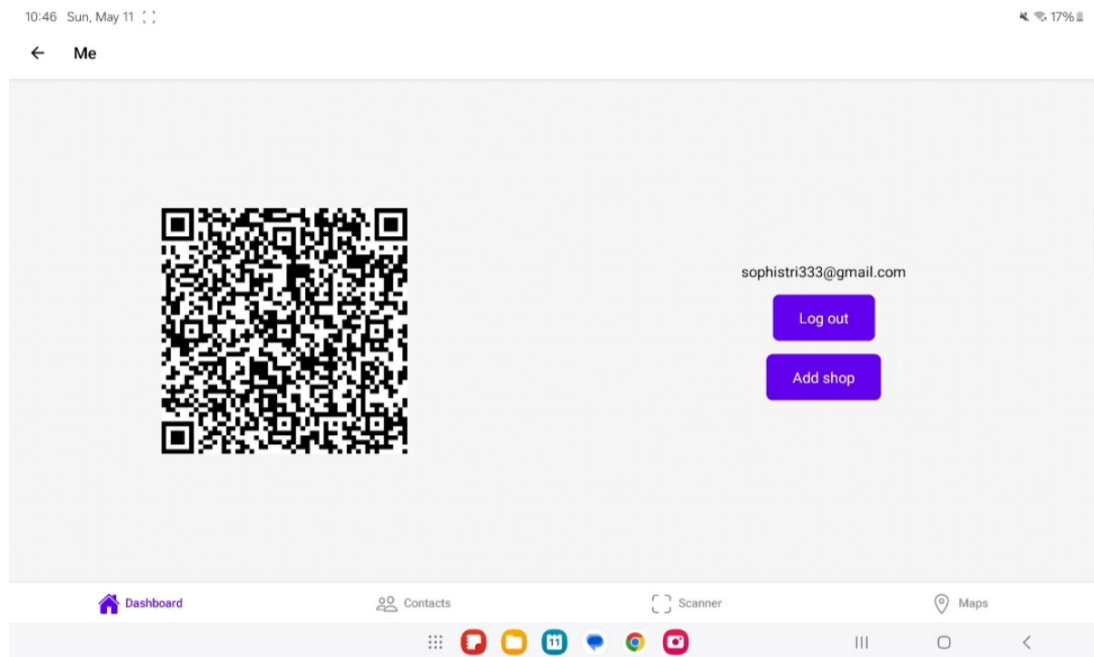


Figure 3.25: Me landscape screen.

### 3.4.6. Other pages

In some other cases due to the limited amount of content and/or the nature of the screen it has not been split left to right but the layout simply widens in a responsive manner.

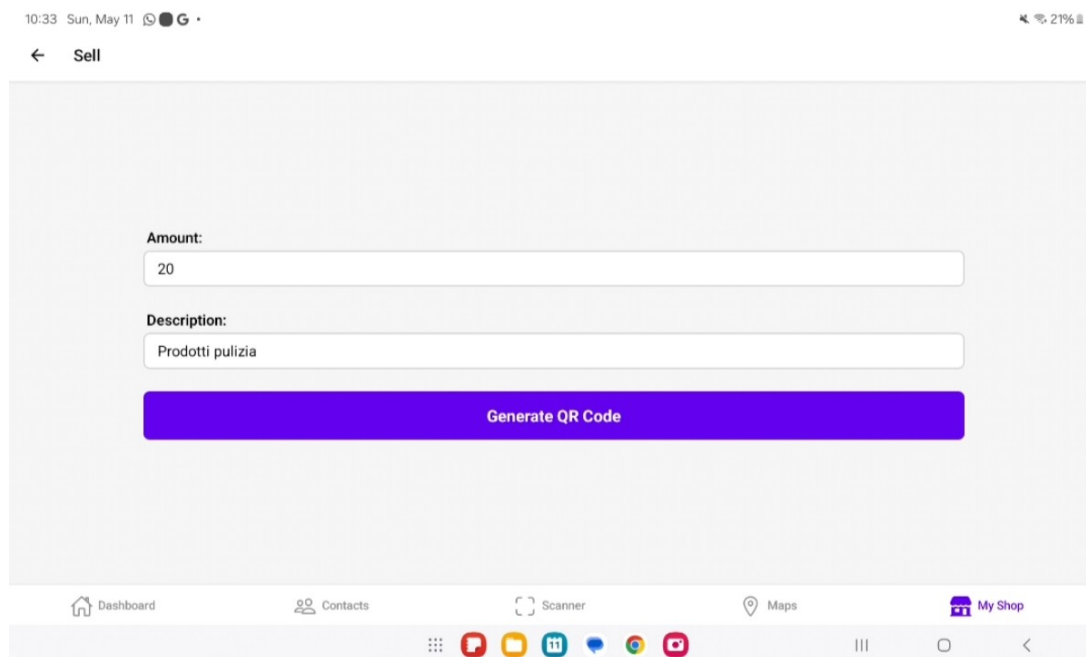


Figure 3.26: Sell landscape screen.

## 3.5. Dark mode

Our application also supports dark mode, providing a visually comfortable experience in low-light environments. When dark mode is enabled, all interface elements—including tabs, modals, buttons, and navigation bars—adapt to a darker color palette while preserving readability and accessibility. The dark theme is implemented using a shared design system that dynamically switches styles based on the user’s system preference. This ensures the app has a consistent visual presentation with respect to the other applications. Dark mode support is particularly useful for users who frequently interact with the app at night or in dim settings. It helps reduce eye strain and aligns with modern user expectations for personalization and theme adaptability. We not only adapted the main UI to account for these visual changes: we also developed custom modals and a custom maps theme in order to avoid having elements which wouldn’t be consistent with the dark mode.

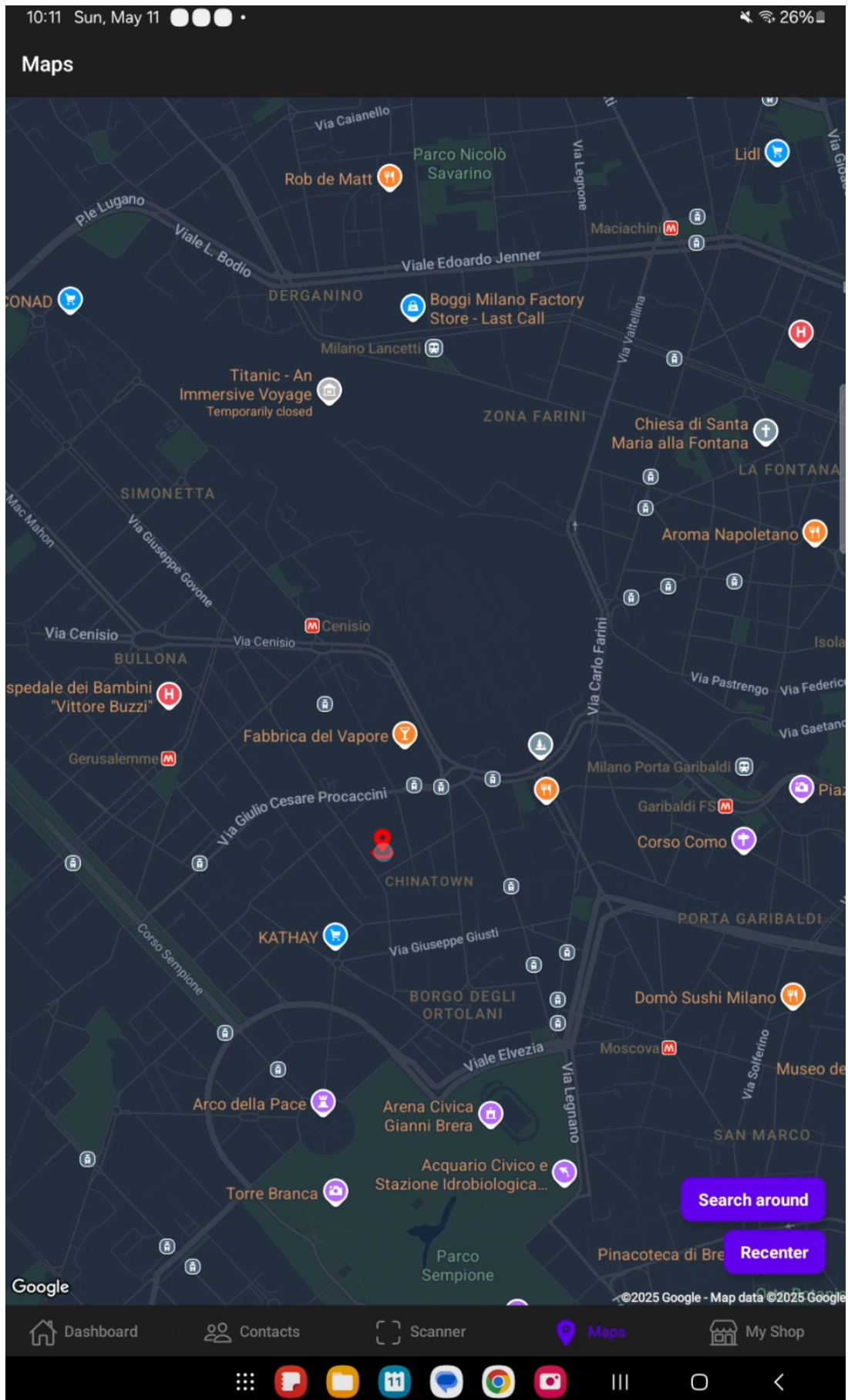


Figure 3.27: Maps dark.

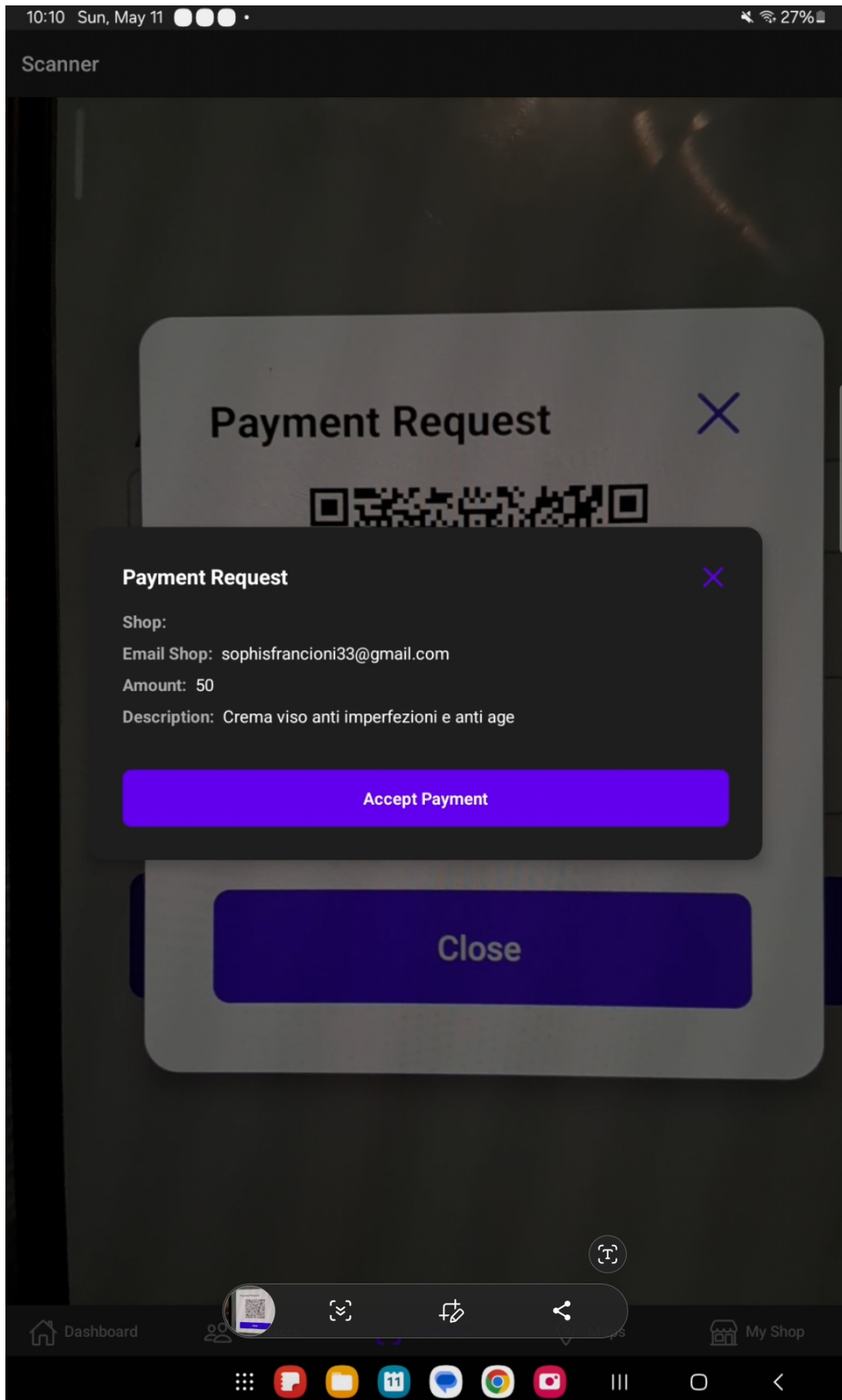


Figure 3.28: Payment request dark.

# 4 | Testing

## 4.1. Testing Campaign

To ensure the reliability, performance, and user satisfaction of the mobile application, a comprehensive and structured testing campaign was designed and executed. This campaign included two main phases: automated testing and user testing. Automated testing was employed to systematically verify the correctness, stability, and consistency of the application's behavior across various components and use cases. User testing, on the other hand, focused on the real-world usage of the app, allowing to collect valuable feedback on usability, design intuitiveness, and overall user experience.

## 4.2. Unit Test

Unit testing was performed to ensure the correctness and of all **helper functions and custom hooks** used throughout the application, **tested in isolation from UI components and external systems**.

| Hook                        | Description                                                                                                         | Unit Tests                                                                                                                                                                              |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useUser</code>        | Hook to manage the user context within the application, it stores user information and actions for login and logout | UT1: test <code>UserContext.Provider</code> functionality<br>UT2: test initial state<br>UT3: test login user<br>UT4: test update user info<br>UT5: test logOut                          |
| <code>useThemeColor</code>  | Hook to manage the application theme                                                                                | UT1: test picking correct color in light mode<br>UT2: test picking correct color in dark mode<br>UT3: test picking fallback color<br>UT4: test picking light and dark mode sequentially |
| <code>useOrientation</code> | Hook to manage the physical mobile orientation                                                                      | UT1: test initial orientation<br>UT2: test updating of orientation                                                                                                                      |
| <code>useIsLandscape</code> | Hook to manage the landscape layout                                                                                 | UT1: test landscape right orientation<br>UT2: test landscape left orientation<br>UT3: test portrait up orientation<br>UT4: test portrait down orientation                               |

| Hook                  | Description                                                                                | Unit Tests                                                                                                                                                           |
|-----------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| usePermission-Handler | Hook to manage the permissions for physical devices functionalities as camera and location | UT1: test camera permissions granted<br>UT2: test camera permissions not granted<br>UT3: test location permissions granted<br>UT4: test location permissions granted |

**Table 4.1: Unit test coverage for custom React Hooks**

**Unit test overview:** the unit tests that have been implemented demonstrate comprehensive coverage, with a strong focus on both functionality and edge cases. Specifically, there are *20 tests dedicated to the hooks*, and *17 tests tests dedicated to the helper functions*, covering a wide range of scenarios to guarantee their correct operation across different use cases.

|              | Statements | Branches | Functions | Lines   |
|--------------|------------|----------|-----------|---------|
| Hooks        | 96.05%     | 78.94%   | 94.73%    | 97.18%  |
| Helper func. | 100.00%    | 100.00%  | 100.00%   | 100.00% |

**Table 4.2: Unit test results**

### 4.3. Component Test

Component testing has been carefully designed to thoroughly assess the behavior of individual components integral to the entire application. Both the UI aspect—assessing the correct rendering of components—and UI interaction—simulating user actions to assess correct component behavior—have been tested. Furthermore, component tests are critical because the application’s overall performance and reliability are heavily dependent on the correct behavior of these building blocks

| Folder | Component        | Test Cases                                                                                                                           |
|--------|------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| base   | AddContactButton | CT1: Correct rendering of the component<br>CT2: Correct interaction with button                                                      |
|        | BaseError        | CT1: Correct rendering with error message<br>CT2: Correct rendering when no message is provided                                      |
|        | BaseLabel        | CT1: Correct rendering of the component                                                                                              |
|        | BaseText         | CT1: Correct rendering of the component<br>CT2: Correct rendering with light theme                                                   |
|        | LabelRow         | CT1: Correct rendering of component with children                                                                                    |
|        | LabelValueRow    | CT1: Correct rendering with light theme<br>CT2: Correct rendering with dark theme<br>CT3: Correct rendering of the component content |

| Folder     | Component               | Test Cases                                                                                                                                                                                                                                                    |
|------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| input      | InputDocuments          | CT1: Correct rendering of initial documents<br>CT2: Correct rendering of unnamed document<br>CT3: Correct interaction with the document picker<br>CT4: Correct interaction to remove a document                                                               |
|            | InputLocation           | CT1: Correct rendering when a location provided<br>CT2: Correct behaviour when a location is not provided<br>CT3: Correct set of initial map location<br>CT4: Correct update of map location                                                                  |
|            | InputText               | CT1: Correc rendering of input component<br>CT2: Correct set of initial value of input<br>CT3: Correct interaction with input component<br>CT4: Correct rendering of error message                                                                            |
| modals     | AppModal                | CT1: Correct rendering of the modal content<br>CT2: Correct behavior of the modal when not visible<br>CT3: Correct interaction with close button<br>CT4: Correct interaction with action button                                                               |
|            | ContactRequest-Modal    | CT1: Correct rendering of the modal content<br>CT2: Correct behavior of the modal when not visible<br>CT3: Correct interaction with close button<br>CT4: Correct interaction with action button                                                               |
|            | MessageModal            | CT1: Correct rendering of the component                                                                                                                                                                                                                       |
|            | PaymentRequest-Modal    | CT1: Correct rendering of the modal content<br>CT2: Correct behavior of the modal when not visible<br>CT3: Correct interaction with close button                                                                                                              |
|            | PermissionRequest-Modal | CT1: Correct rendering of the modal content<br>CT2: Correct behavior of the modal when not visible<br>CT3: Correct interaction with close button and open app settings (iOS)<br>CT4: Correct interaction with close button and open app settings (Android)    |
|            | TransactionDetail-Modal | CT1: Correct rendering of the modal content<br>CT2: Correct behavior of the modal when not visible<br>CT3: Correct interaction with close button<br>CT4: Correct navigation when transactor isn't a shop<br>CT5: Correct navigation when transactor is a shop |
| list-ables | ContactListable         | CT1: Correct rendering of component content<br>CT2: Corrcet interaction with component button                                                                                                                                                                 |
|            | MessageListable         | CT1: Correct rendering of component content<br>CT2: Correct rendering of component when amount is negative                                                                                                                                                    |
|            | TransactionListable     | CT1: Correct render of component content<br>CT2: Correct opening of modal<br>CT3: Correct closing of modal<br>CT4: Correct rendering of error message                                                                                                         |

**Table 4.3: Component test cases overview**

**Component test overview:** The component tests implemented exhibit thorough and well-structured coverage, with a strong emphasis on both the accurate rendering of UI elements and the reliable simulation of user interactions. These tests target foundational components that constitute the core structure of the application. As such, verifying their visual correctness and behavioral integrity is essential to ensuring consistency, usability, and alignment with the intended user experience.

| Folder    | Statements | Branches | Functions | Lines   |
|-----------|------------|----------|-----------|---------|
| Base      | 88.46%     | 43.75%   | 85.71%    | 88.46%  |
| Input     | 96.15%     | 73.33%   | 100.00%   | 100.00% |
| Modals    | 100.00%    | 61.11%   | 100.00%   | 100.00% |
| Listables | 100.00%    | 61.11%   | 100.00%   | 100.00% |
| Section   | 87.69%     | 69.09%   | 71.42%    | 89.65%  |

Table 4.4: Component test results

## 4.4. Integration Test

The integration test was developed based on unit and component tests, whose correctness is crucial for the integration test to be effective.

The integration test was designed to validate each page of the application across three key dimensions:

- **Correctness of data and component rendering**, ensuring that all dynamic content and UI elements are displayed accurately and as expected.
- **Correctness of user interactions**, by simulating realistic user behavior and verifying the appropriate responses from the UI components.
- **Correctness of navigation**, checking that all allowed navigation paths lead to the intended destinations and preserve the expected application state.

| Section   | Page  | Test suite  | Test Cases                                                                                                                                                                                      |
|-----------|-------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dashboard | index | Render      | IT1: Correct call of functions on mount<br>IT2: Correct rendering components and data on mount                                                                                                  |
|           |       | Interaction | IT1: Interaction with 'Withdrawal'<br>IT2: Interaction with 'Deposit'<br>IT3: Interaction with single Transaction record which opens a modal<br>IT4: Interaction with 'Me' button to navigation |
|           |       | Navigation  | IT1: Navigate to 'Me' page                                                                                                                                                                      |

| Section  | Page  | Test suite  | Test Cases                                                                                                                                                                                                                                                                                                 |
|----------|-------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          | me    | Render      | IT1: Correct rendering of user info and qr code<br>IT2: Correct rendering of 'Add shop' button when user has not a Shop<br>IT3: Do not display the 'Add shop' button when user has already a Shop                                                                                                          |
|          |       | Interaction | IT1: Correct interaction with 'Add Shop'                                                                                                                                                                                                                                                                   |
|          |       | Navigation  | IT1: Navigation to the 'Add' page                                                                                                                                                                                                                                                                          |
|          | add   | Render      | IT1: Correct rendering of components when location permissions allowed<br>IT2: Correct rendering of components when location permissions denied                                                                                                                                                            |
|          |       | Interaction | IT1: Correct interaction with 'Create shop' text input fields<br>IT2: Correct interaction with 'upload document' field<br>IT3: Correct creation of 'Shop'<br>IT4: Correct rendering of error when error on database interaction<br>IT5: Correct interaction with components to navigate to the 'Shop' page |
|          |       | Navigation  | IT1: Navigation to the 'Shop' page                                                                                                                                                                                                                                                                         |
| maps     | index | Render      | IT1: Correct rendering of user position<br>IT2: Correct rendering of shop marker                                                                                                                                                                                                                           |
|          |       | Interaction | IT1: Correct interaction with 'Recenter'                                                                                                                                                                                                                                                                   |
| contacts | index | Render      | IT1: Correct rendering of contacts in vertical mode<br>IT2: Correct rendering of contacts in landscape mode                                                                                                                                                                                                |
|          |       | Interaction | IT1: Correct interaction with 'Contact' to open chat in landscape mode<br>IT2: Correct interaction with 'Contact' to open chat in vertical mode<br>IT3: Correct interaction with 'Add Contact' button in vertical mode<br>IT4: Correct interaction with 'Add Contact' button in landscape mode             |
|          |       | Navigation  | IT1: Navigate to 'Add' page                                                                                                                                                                                                                                                                                |
|          | add   | Render      | IT1: Correct render of user info and qr code<br>IT2: Render of 'Add shop' button when user has not a Shop<br>IT3: Do not display the 'Add shop' button when user has already a Shop                                                                                                                        |
|          |       | Interaction | IT1: Correct interaction with 'Add Shop'                                                                                                                                                                                                                                                                   |
|          |       | Navigation  | IT1: Navigation to the 'Add' page                                                                                                                                                                                                                                                                          |
|          | [id]  | Render      | IT1: Correct rendering of messages                                                                                                                                                                                                                                                                         |

[id]

| Section | Page  | Test suite  | Test Cases                                                                                                                                                                                                                                         |
|---------|-------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |       | Interaction | IT1: Correct sending of message<br>IT2: Correct displaying of error                                                                                                                                                                                |
| scanner | index | Render      | IT1: Correct rendering of camera                                                                                                                                                                                                                   |
|         |       | Interaction | IT1: Correct scan of 'Add Contact' qr-code<br>IT2: Correct scan of 'Payment request' qr-code<br>IT3: Correct display of error when a payment request has expired<br>IT4: Correct display of error when a payment request has been already accepted |
| shop    | index | Render      | IT1: Correct calling of 'on mount' functions<br>IT2: Correct rendering of components and data retrieved from 'on mount' functions                                                                                                                  |
|         |       | Interaction | IT1: Interaction with 'move to personal account' button<br>IT2: Interaction with 'Sell' button                                                                                                                                                     |
|         |       | Navigation  | IT1: Navigate to 'Sell' page                                                                                                                                                                                                                       |
|         | sell  | Render      | IT1: Correct rendering of input fields                                                                                                                                                                                                             |
|         |       | Interaction | IT1: Opening of the modal with qr-code when the button 'Sell' is pressed<br>IT2: Closing of the modal<br>IT3: Listener functionality to display a message when the payment is accepted                                                             |

**Table 4.5: Integration test overview across all sections**

**Integration test overview:** The integration tests provide comprehensive coverage across all application pages, focusing on various critical aspects, covering aspects such as component rendering, user interactions, and smooth navigation across screens. This final phase of the automated testing campaign builds upon the solid foundation established by unit and component tests, whose correctness is essential to ensure overall reliability and robustness of the application.

| Section              | Statements | Branches | Functions | Lines  |
|----------------------|------------|----------|-----------|--------|
| Dashboard            | 77.06%     | 55.55%   | 69.44%    | 80.8%  |
| Contacts             | 65.21%     | 59.37%   | 60.86%    | 70.31% |
| Contacts/<br>contact | 78.57%     | 50.00%   | 100.00%   | 78.57% |
| Maps                 | 66.66%     | 56.66%   | 60.00%    | 70.76% |
| Scanner              | 79.22%     | 63.15%   | 66.66%    | 80.26% |
| Shop                 | 67.77%     | 58.06%   | 57.69%    | 72.28% |

**Table 4.6: Integration test results**

# 4.5. User Test

User testing was conducted to evaluate the usability and efficiency of key features within the app. A total of 8 participants, varying in age and confidence with technology, were involved in this study. The test was structured around two categories of tasks: *individual tasks* and *cooperative tasks*.

All tasks began from a consistent starting point: the app’s main dashboard page and the user interface was displayed using the app’s "day" theme. By controlling both the starting point and the visual theme, we ensured that the test results more accurately reflect usability differences.

## Individual Tasks

The following tasks were completed by users individually to assess their personal interaction with key app features such as sending money, creating shops, and withdrawing funds.

### Send Money from One Contact to Another

Each user sent money from one contact to another using the app, to test the ease of the transfer feature.

| UserId | Age | Tech Affinity | Time (sec) |
|--------|-----|---------------|------------|
| U001   | 22  | High          | 12         |
| U002   | 47  | Medium        | 35         |
| U003   | 24  | High          | 16         |
| U004   | 52  | Low           | 62         |
| U005   | 49  | Medium        | 28         |
| U006   | 23  | High          | 11         |
| U007   | 54  | Low           | 67         |
| U008   | 22  | High          | 13         |

Table 4.7: Money Transfer Completion Data

### Create a Shop

Users created a shop within the app to evaluate the setup workflow and any potential blockers.

| UserId | Age | Tech Affinity | Time (sec) |
|--------|-----|---------------|------------|
| U001   | 22  | High          | 50         |
| U002   | 47  | Medium        | 80         |
| U003   | 24  | High          | 42         |
| U004   | 52  | Low           | 105        |
| U005   | 49  | Medium        | 73         |
| U006   | 23  | High          | 48         |
| U007   | 54  | Low           | 118        |
| U008   | 22  | High          | 41         |

**Table 4.8: Shop Creation Completion Data**

### Withdraw Money from the Shop

Users withdrew money from their created shop accounts, testing the withdrawal process and clarity of the user interface.

| UserId | Age | Tech Affinity | Time (sec) |
|--------|-----|---------------|------------|
| U001   | 22  | High          | 6          |
| U002   | 47  | Medium        | 15         |
| U003   | 24  | High          | 8          |
| U004   | 52  | Low           | 54         |
| U005   | 49  | Medium        | 18         |
| U006   | 23  | High          | 7          |
| U007   | 54  | Low           | 62         |
| U008   | 22  | High          | 7          |

**Table 4.9: Shop Withdrawal Completion Data**

### Cooperative Tasks

The following tasks were completed in pairs to evaluate how users interact together, including using the scanner feature and completing shop payments collaboratively.

#### Add Contact Using Scanner

Users worked in pairs to add a contact by scanning a QR code. The goal was to evaluate how intuitive and smooth the scanning feature is.

| UserId (Pair) | Age     | Tech Affinity   | Time (sec) |
|---------------|---------|-----------------|------------|
| U001 + U003   | 22 + 24 | High + High     | 25         |
| U002 + U005   | 47 + 49 | Medium + Medium | 67         |
| U004 + U007   | 52 + 54 | Low + Low       | 85         |
| U006 + U008   | 23 + 22 | High + High     | 32         |

**Table 4.10: Contact Addition Completion Data**

Pay at the Shop

Users worked in pairs to complete a payment at a shop, testing the user flow and scanning/payment interaction.

| UserId (Pair) | Age     | Tech Affinity   | Time (sec) |
|---------------|---------|-----------------|------------|
| U001 + U003   | 22 + 24 | High + High     | 44         |
| U002 + U005   | 47 + 49 | Medium + Medium | 63         |
| U004 + U007   | 52 + 54 | Low + Low       | 85         |
| U006 + U008   | 23 + 22 | High + High     | 40         |

Table 4.11: Shop Payment Completion Data

Conclusions

The task completion data highlights distinct patterns linked to users’ technological affinity and age. High-affinity users, all between 22 and 24 years old, completed tasks quickly and consistently across both individual and cooperative scenarios, demonstrating confidence and fluency with mobile app interactions.

In contrast, older users with medium and low tech affinity (ages 45–55) exhibited longer completion times. Most of the delays observed in this group stemmed from spending more time reading on-screen information carefully, which may be partly influenced by varying levels of English language comprehension. This behavior was especially noticeable during initial exposure to tasks or less familiar screens.

Despite occasional wandering through the interface, none of the users became stuck at any point. This can be attributed to the app’s low navigation depth and overall simplicity, which helped users remain oriented and recover easily from navigation errors. The results suggest that while the app is already accessible, small enhancements in clarity and visual guidance could further improve the experience for older or less tech-savvy users.

User Feedback

During testing, users provided a range of suggestions aimed at improving usability and extending functionality. Several recurring themes emerged across feedback sessions:

- **Language Options:** Multiple users requested support for additional languages, noting that understanding certain instructions in English was challenging.
- **Transaction Management:** Users expressed interest in being able to filter and sort transactions, such as by date, amount, or type, to make financial history easier to navigate.
- **Shop Editing:** Participants highlighted the need for the ability to edit an existing shop’s details, such as name or location, after creation.
- **Contact Management:** Suggestions included the ability to delete contacts and to rename them for better personal organization.

- **Contact Ordering:** Several users recommended ordering the contact list based on the last time money was sent to a contact, to surface frequently used recipients.

As this is a project with limited scope and resources, we had to draw the line somewhere. While these valuable suggestions will not be implemented in the current version, they provide important insights.